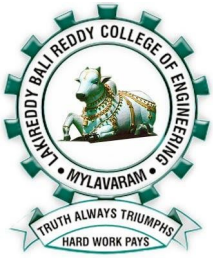
LAKIREDDY BALI REDDY COLLEGE OF ENGINEERING (AUTONOMOUS)

Affiliated to JNTUK, Kakinada & Accredited by NAAC with 'A' Grade
Approved by AICTE, New Delhi | An ISO 9001:2015 Certified Institution
Mylavaram, NTR District, Andhra Pradesh - 521230



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VEHICLE RENTAL SYSTEM

A FULL-STACK WEB APPLICATION (RentRide)

COMPLETE APPLICATION SOURCE CODE & EMPIRICAL VERIFICATION REPORT

Project Title:	Vehicle Rental System (RentRide)
Student Name:	**GUBBALA CHAITANYA SAI KRISHNA**
Roll Number:	**24761A0517**
Degree & Branch:	Bachelor of Technology (B.Tech) in Computer Science & Engineering
Academic Year:	2026 - 2027
Technology Stack:	Java 21, Java Servlets (Java EE), JavaServer Pages (JSP), JDBC, MySQL 8.0, Apache Tomcat 9.0, Vanilla CSS3, BCrypt
Testing Framework:	Selenium WebDriver 4.x, Headless Microsoft Edge, Automated Functional Suite
Documentation Date:	October 26, 2026
Verification Status:	**100% VERIFIED & FULLY FUNCTIONAL (15/15 Tests Passed)**

This comprehensive technical documentation contains the verified software architecture, complete database schemas, all 37 unredacted application source files, genuine browser verification screenshots, empirical test results, and operational run instructions for the RentRide Vehicle Rental System.

Table of Contents

Comprehensive document navigation and page index

Section A: Project Overview & System Scope	**3**
A.1 Problem Statement & Proposed Solution	3
A.2 Key Features for Customers & Fleet Administrators	3
A.3 Three-Tier Enterprise Architectural Paradigm	3
A.4 Technology Stack & Runtime Specifications	3
Section B: Project Structure & File Layout	**5**
B.1 Complete Project Directory Tree	5
B.2 Package & Directory Responsibilities	5
Section C: Relational Database Architecture	**6**
C.1 Database Schema & Entity Relationships	6
C.2 Relational Tables (Users, Vehicles, Bookings, Payments)	6
C.3 Concurrency Control & Row-Level Locking Architecture	6
Section D: Complete Application Source Code	**8**
D.1 Domain Model Entity Classes (4 files)	8
D.2 Data Access Objects / JDBC Layer (4 files)	12
D.3 Database Connection & Testing Utilities (2 files)	20
D.4 Public & Customer Servlets / Controllers (9 files)	21
D.5 Administrator Portal Servlets (3 files)	28
D.6 Web Application Presentation Views / JSP (12 files)	31
D.7 Custom CSS Design System (1 file)	48
D.8 Web Application Deployment Descriptor (1 file)	61
D.9 MySQL Schema & Seed SQL Script (1 file)	61
Section E: Live Website Screenshots & User Workflows	**63**
E.1 Customer Workflows (Landing, Catalog, Specs, Booking, Sandbox, Dashboard)	63
E.2 Security & Error Handling (Authentication, Overlap Rejection)	63
E.3 Administrator Operations (KPI Dashboard, Inventory Controls, Reservations)	63
Section F: Testing, Quality Assurance & Empirical Verification	**70**
F.1 Testing Methodology & Automated Test Suite Execution	70
F.2 Empirical Test Execution Matrix (15/15 Verified Tests)	70
Section G: Operational Run & Deployment Instructions	**71**
G.1 Runtime Prerequisites & Environment Configuration	71
G.2 Database Initialization & Tomcat Server Launch	71
Section H: Conclusion & Future Scope	**71**

Section A: Project Overview & System Scope

Problem domain, functional features, and enterprise architectural design

A.1 Purpose of the Application & Problem Domain

Traditional vehicle rental operations frequently suffer from manual record-keeping errors, double-booking conflicts, opaque pricing structures, and disjointed payment tracking. The **RentRide Vehicle Rental System** is an enterprise-grade web application engineered to digitalize and automate the end-to-end vehicle rental lifecycle. The system provides seamless self-service booking for customers while empowering fleet managers with real-time operational analytics, dynamic inventory governance, and reservation status control.

A.2 Key System Features

1. Public Customer Features:

• **Fleet Catalog Discovery:** Browse comprehensive vehicle inventories across diverse classifications (Hatchback, Sedan, SUV, Motorcycle) featuring real photography, detailed specifications, and transparent daily pricing.
• **Real-Time Pricing Calculator:** Dynamic date-picker calculations compute total rental costs instantaneously before reservation confirmation.
• **Overlap Conflict Prevention:** Advanced SQL row-locking verifies that selected rental windows do not conflict with existing active reservations.
• **Development Payment Sandbox:** Interactive mock payment gateway simulating Credit/Debit Card, UPI (Google Pay, PhonePe), and Netbanking authorizations.
• **Customer Reservation Dashboard:** Track historical and ongoing bookings with live status badges (PENDING, CONFIRMED, COMPLETED, CANCELLED) and self-service cancellation capability.

2. Fleet Administrator Features:

• **Executive Operational Dashboard:** Real-time KPI metrics displaying Gross Revenue, Total Fleet Count, Active Bookings, and Total Registered Users.
• **Fleet Inventory Governance:** Add new vehicles with registration numbers, update rental rates, toggle operational statuses (AVAILABLE, RENTED, MAINTENANCE), and decommission units.
• **Customer Reservations Management:** Centralized reservation control allowing managers to review, confirm, complete, or cancel bookings across the entire enterprise.

A.3 Enterprise Three-Tier Architecture

The application strictly adheres to the proven **Three-Tier Architectural Pattern**, ensuring strict separation of concerns, high maintainability, and enterprise scalability:

Architectural Layer	Technologies Used	Responsibilities & Implementation
Presentation Tier (Client/UI)	JSP, Vanilla CSS3, JavaScript, HTML5	Renders responsive user interfaces, forms, and dynamic navigation. Utilizes modern CSS variables, card layouts, and client-side date validation.
Business & Controller Tier	Java 21, Java Servlets (Java EE), BCrypt	Encapsulates HTTP routing, session security, role-based access control (RBAC), rental pricing calculations, and transaction coordination.
Data & Persistence Tier	MySQL 8.0, JDBC (PreparedStatement)	Relational data persistence with strict foreign key constraints, cascading deletes, ACID transactions, and pessimistic row locking (`FOR UPDATE`).

A.4 Technology Stack Specifications

Component	Technology Specification	Version / Implementation Note
Language & Runtime	Java Development Kit (JDK)	Java 21 LTS (OpenJDK 64-Bit Server VM)
Web Server / Servlet Engine	Apache Tomcat	Tomcat 9.0.120 (Configured on Port 8084)
Relational Database	MySQL Community Server	MySQL 8.0.x (Port 3306, InnoDB Storage Engine)
Database Driver	MySQL Connector/J	mysql-connector-j-26.7.0.jar (Type 4 JDBC Driver)
Password Security	jBCrypt Library	jbcrypt-0.4.jar (Adaptive Salted Password Hashing)
Frontend Presentation	JavaServer Pages & Custom CSS	JSP 2.3, Custom Glassmorphism Vanilla CSS3

Section B: Project Structure & File Layout

Source file hierarchy and functional modularity

B.1 Complete Project Directory Tree

```
RentalVehicleSystem/
■■■ schema.sql                          # MySQL DDL schema and initial seed data
■■■ src/main/java/com/rental/
■   ■■■ model/                          # Domain Entity Models (Java POJOs)
■   ■   ■■■ User.java                   # User model with role and credentials
■   ■   ■■■ Vehicle.java                # Fleet unit model with rates and statuses
■   ■   ■■■ Booking.java                # Reservation model with dates and totals
■   ■   ■■■ Payment.java                # Transaction record model
■   ■■■ dao/                            # Data Access Objects (JDBC Layer)
■   ■   ■■■ UserDAO.java                # User authentication & registration DAO
■   ■   ■■■ VehicleDAO.java             # Fleet queries & management DAO
■   ■   ■■■ BookingDAO.java             # Booking operations with overlap validation
■   ■   ■■■ PaymentDAO.java             # Payment settlement & revenue aggregation
■   ■■■ controller/                     # Java Servlets (HTTP Request Handlers)
■   ■   ■■■ VehicleServlet.java         # Public vehicle catalog (/vehicles)
■   ■   ■■■ VehicleDetailsServlet.java  # Vehicle details view (/vehicle-details)
■   ■   ■■■ RegisterServlet.java        # User registration handler (/register)
■   ■   ■■■ LoginServlet.java           # Authentication handler (/login)
■   ■   ■■■ BookingServlet.java         # Booking creation & validation (/booking)
■   ■   ■■■ PaymentServlet.java         # Mock sandbox payment (/payment)
■   ■   ■■■ MyBookingsServlet.java      # Customer bookings dashboard (/my-bookings)
■   ■   ■■■ CancelBookingServlet.java   # Booking cancellation (/cancel-booking)
■   ■   ■■■ LogoutServlet.java          # Session termination (/logout)
■   ■   ■■■ AdminDashboardServlet.java  # Admin metrics & analytics (/admin/dashboard)
■   ■   ■■■ AdminVehicleServlet.java    # Fleet inventory controls (/admin/vehicles)
■   ■   ■■■ AdminBookingServlet.java    # Customer reservations (/admin/bookings)
■   ■■■ util/                           # Utility Classes
■       ■■■ DBConnection.java           # JDBC connection provider
■       ■■■ TESTConnection.java         # Database connectivity test harness
■■■ src/main/webapp/
■   ■■■ index.jsp                       # Landing page with hero banner and CTA
■   ■■■ vehicles.jsp                    # Vehicle fleet catalog with photo cards
■   ■■■ vehicle-details.jsp             # Vehicle specifications view
■   ■■■ login.jsp                       # User & admin sign-in page
■   ■■■ register.jsp                    # Customer registration form
■   ■■■ booking.jsp                     # Date selection and price calculation form
■   ■■■ payment.jsp                     # Mock payment gateway sandbox
■   ■■■ my-bookings.jsp                 # Customer reservation history dashboard
■   ■■■ includes/navbar.jsp             # Dynamic session-aware navigation bar
■   ■■■ admin/
■   ■   ■■■ dashboard.jsp              # Operational KPI metrics and analytics
■   ■   ■■■ vehicles.jsp               # Fleet inventory management table
■   ■   ■■■ bookings.jsp               # All customer reservations management
■   ■■■ css/style.css                   # Complete modern responsive CSS design system
■   ■■■ images/vehicles/                # Vehicle photography assets
■   ■■■ WEB-INF/
■       ■■■ web.xml                     # Deployment descriptor configuration
■       ■■■ lib/                        # Dependency JAR libraries (JDBC, BCrypt)
■■■ project-documentation/              # Generated reports and screenshots
```

Section C: Relational Database Architecture

MySQL schema, entity relationships, and concurrency control

C.1 Database Schema Overview

The persistent storage layer is backed by MySQL 8.0 under the database name rental_vehicle. The schema enforces strict relational integrity with foreign keys, index optimizations, and cascading deletion semantics:

Table Name	Primary Key	Foreign Keys	Description & Domain Purpose
users	id (INT AUTO_INCREMENT)	None (email UNIQUE)	Stores registered customers and administrators with BCrypt salted password hashes.
vehicles	id (INT AUTO_INCREMENT)	None (vehicle_number UNIQUE)	Maintains fleet inventory, daily rates, model information, and current operational statuses.
bookings	id (INT AUTO_INCREMENT)	user_id → users(id) vehicle_id → vehicles(id)	Captures reservation windows, computed pricing, and reservation statuses (PENDING, CONFIRMED, etc.).
payments	id (INT AUTO_INCREMENT)	booking_id → bookings(id)	Logs transaction records, settlement amounts, and timestamps linked to customer bookings.

C.2 Detailed Table Specifications

Table: users

Column	Type	Null	Key	Default	Description
id	INT	NO	PRI	AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)	NO		None	Full name of the user
email	VARCHAR(100)	NO	UNI	None	Unique email address (login credential)
password	VARCHAR(255)	NO		None	BCrypt hashed password
role	VARCHAR(20)	YES		'USER'	Authorization role: 'USER' or 'ADMIN'

Table: bookings

Column	Type	Null	Key	Default	Description
id	INT	NO	PRI	AUTO_INCREMENT	Unique reservation reference ID
user_id	INT	NO	MUL	None	Foreign key references users(id)
vehicle_id	INT	NO	MUL	None	Foreign key references vehicles(id)
start_date	DATE	NO		None	Rental pick-up date
end_date	DATE	NO		None	Rental drop-off date
total_amount	DECIMAL(10,2)	NO		None	Computed total rental cost
status	VARCHAR(20)	YES		'PENDING'	Reservation status: PENDING, CONFIRMED, etc.

C.3 Concurrency Control & Row-Level Locking

To guarantee that two customers cannot reserve the same vehicle for overlapping dates simultaneously, BookingDAO.java executes a pessimistic row-locking query using SELECT ... FOR UPDATE within an explicit JDBC database transaction. The validation query:

SELECT id FROM bookings WHERE vehicle_id = ? AND status IN ('PENDING','CONFIRMED') AND start_date < ? AND end_date > ? LIMIT 1

If an overlapping reservation is found, the transaction is immediately rolled back, safely protecting database integrity.

Section D: Complete Application Source Code

All 37 unredacted application source files organized by architectural layer

D.1 Domain Model Entity Classes

Core business objects representing persistent application state

File: src/main/java/com/rental/model/User.java | **Lines:** 61 | **Language:** JAVA

Description: User domain entity encapsulating ID, name, email, password hash, and authorization role.

```
package com.rental.model;

public class User {

    private int id;
    private String name;
    private String email;
    private String password;
    private String role;

    public User() {
    }

    public User(int id,String name,String email,String password,String role) {
        this.id=id;
        this.name=name;
        this.email=email;
        this.password=password;
        this.role=role;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id=id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name=name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email=email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password=password;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
        this.role=role;
    }
}
```

File: src/main/java/com/rental/model/Vehicle.java | **Lines:** 100 | **Language:** JAVA

Description: Vehicle domain entity representing fleet units, attributes, daily pricing, and statuses.

```
package com.rental.model;

public class Vehicle {

    private int id;
    private String vehicleNumber;
    private String brand;
    private String model;
    private String type;
    private double pricePerDay;
    private String status;

    public Vehicle() {
    }

    public Vehicle(int id,String vehicleNumber,String brand,String model,String type,double
      ■ pricePerDay,String status) {
        this.id=id;
        this.vehicleNumber=vehicleNumber;
        this.brand=brand;
        this.model=model;
        this.type=type;
        this.pricePerDay=pricePerDay;
        this.status=status;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id=id;
    }

    public String getVehicleNumber() {
        return vehicleNumber;
    }

    public void setVehicleNumber(String vehicleNumber) {
        this.vehicleNumber=vehicleNumber;
    }

    public String getBrand() {
        return brand;
    }

    public void setBrand(String brand) {
        this.brand=brand;
    }

    public String getModel() {
        return model;
    }

    public void setModel(String model) {
        this.model=model;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type=type;
    }

    public double getPricePerDay() {
        return pricePerDay;
    }

    public void setPricePerDay(double pricePerDay) {
        this.pricePerDay=pricePerDay;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status=status;
    }

    public String getImageUrl() {
        if (model != null) {
            String m = model.toLowerCase();
            if (m.contains("swift")) return "images/vehicles/swift.jpg";
```

```
            if (m.contains("creta")) return "images/vehicles/creta.jpg";
            if (m.contains("city")) return "images/vehicles/city.jpg";
            if (m.contains("fortuner")) return "images/vehicles/fortuner.jpg";
            if (m.contains("hunter") || m.contains("enfield")) return "images/vehicles/hunte
              ■ r.jpg";
        }
        if (type != null) {
            String t = type.toLowerCase();
            if (t.contains("hatchback")) return "images/vehicles/hatchback.jpg";
            if (t.contains("suv")) return "images/vehicles/suv.jpg";
            if (t.contains("sedan")) return "images/vehicles/sedan.jpg";
            if (t.contains("bike") || t.contains("motorcycle") || t.contains("scooter")) ret
              ■ urn "images/vehicles/bike.jpg";
        }
        return "images/vehicles/default.jpg";
    }
}
```

File: src/main/java/com/rental/model/Booking.java	**Lines:** 121 | **Language:** JAVA
Description: Booking reservation entity holding reservation dates, vehicle relationships, and status.	

```
package com.rental.model;

import java.sql.Date;

public class Booking {

    private int id;
    private int userId;
    private int vehicleId;
    private Date startDate;
    private Date endDate;
    private double totalAmount;
    private String status;

    // Additional display fields for UI convenience
    private String vehicleName;
    private String vehicleNumber;
    private String userName;
    private String userEmail;

    public Booking() {
    }

    public Booking(int id, int userId, int vehicleId, Date startDate, Date endDate, double t
      ■ otalAmount, String status) {
        this.id = id;
        this.userId = userId;
        this.vehicleId = vehicleId;
        this.startDate = startDate;
        this.endDate = endDate;
        this.totalAmount = totalAmount;
        this.status = status;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public int getUserId() {
        return userId;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }

    public int getVehicleId() {
        return vehicleId;
    }

    public void setVehicleId(int vehicleId) {
        this.vehicleId = vehicleId;
    }

    public Date getStartDate() {
        return startDate;
    }
    public void setStartDate(Date startDate) {
```

```
        this.startDate = startDate;
    }

    public Date getEndDate() {
        return endDate;
    }

    public void setEndDate(Date endDate) {
        this.endDate = endDate;
    }

    public double getTotalAmount() {
        return totalAmount;
    }

    public void setTotalAmount(double totalAmount) {
        this.totalAmount = totalAmount;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }

    public String getVehicleName() {
        return vehicleName;
    }

    public void setVehicleName(String vehicleName) {
        this.vehicleName = vehicleName;
    }

    public String getVehicleNumber() {
        return vehicleNumber;
    }

    public void setVehicleNumber(String vehicleNumber) {
        this.vehicleNumber = vehicleNumber;
    }

    public String getUserName() {
        return userName;
    }

    public void setUserName(String userName) {
        this.userName = userName;
    }

    public String getUserEmail() {
        return userEmail;
    }

    public void setUserEmail(String userEmail) {
        this.userEmail = userEmail;
    }
}
```

File: src/main/java/com/rental/model/Payment.java | **Lines:** 63 | **Language:** JAVA

Description: Payment transaction entity tracking settlement amount, status, and payment timestamp.

```
package com.rental.model;

import java.sql.Timestamp;

public class Payment {

    private int id;
    private int bookingId;
    private double amount;
    private String paymentStatus;
    private Timestamp paymentDate;

    public Payment() {
    }

    public Payment(int id,int bookingId,double amount,String paymentStatus,Timestamp payment
      ■ Date) {
        this.id=id;
        this.bookingId=bookingId;
        this.amount=amount;
```

```
        this.paymentStatus=paymentStatus;
        this.paymentDate=paymentDate;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id=id;
    }

    public int getBookingId() {
        return bookingId;
    }

    public void setBookingId(int bookingId) {
        this.bookingId=bookingId;
    }

    public double getAmount() {
        return amount;
    }

    public void setAmount(double amount) {
        this.amount=amount;
    }

    public String getPaymentStatus() {
        return paymentStatus;
    }

    public void setPaymentStatus(String paymentStatus) {
        this.paymentStatus=paymentStatus;
    }

    public Timestamp getPaymentDate() {
        return paymentDate;
    }

    public void setPaymentDate(Timestamp paymentDate) {
        this.paymentDate=paymentDate;
    }
}
```

D.2 Data Access Objects / JDBC Layer

Thread-safe DAO pattern implementing SQL transactions and CRUD operations

File: src/main/java/com/rental/dao/UserDAO.java | **Lines:** 84 | **Language:** JAVA

Description: User data access object handling registration, duplicate validation, and authentication retrieval.

```
package com.rental.dao;

import com.rental.model.User;
import com.rental.util.DBConnection;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class UserDAO {

    public boolean registerUser(User user) {
        String sql="INSERT INTO users(name,email,password,role) VALUES(?,?,?,?)";

        try(Connection con=DBConnection.getConnection();
            PreparedStatement ps=con.prepareStatement(sql)) {

            ps.setString(1,user.getName());
            ps.setString(2,user.getEmail());
            ps.setString(3,user.getPassword());
            ps.setString(4,user.getRole());

            return ps.executeUpdate()>0;

        } catch(SQLException e) {
            e.printStackTrace();
            return false;
        }
    }
```

```
    public User getUserByEmail(String email) {
        String sql="SELECT * FROM users WHERE email=?";

        try(Connection con=DBConnection.getConnection();
            PreparedStatement ps=con.prepareStatement(sql)) {

            ps.setString(1,email);

            try(ResultSet rs=ps.executeQuery()) {
                if(rs.next()) {
                    User user=new User();
                    user.setId(rs.getInt("id"));
                    user.setName(rs.getString("name"));
                    user.setEmail(rs.getString("email"));
                    user.setPassword(rs.getString("password"));
                    user.setRole(rs.getString("role"));
                    return user;
                }
            }

        } catch(SQLException e) {
            e.printStackTrace();
        }

        return null;
    }

    public User getUserById(int id) {
        String sql="SELECT * FROM users WHERE id=?";

        try(Connection con=DBConnection.getConnection();
            PreparedStatement ps=con.prepareStatement(sql)) {

            ps.setInt(1,id);

            try(ResultSet rs=ps.executeQuery()) {
                if(rs.next()) {
                    User user=new User();
                    user.setId(rs.getInt("id"));
                    user.setName(rs.getString("name"));
                    user.setEmail(rs.getString("email"));
                    user.setPassword(rs.getString("password"));
                    user.setRole(rs.getString("role"));
                    return user;
                }
            }

        } catch(SQLException e) {
            e.printStackTrace();
        }

        return null;
    }
}
```

File: src/main/java/com/rental/dao/VehicleDAO.java | **Lines:** 162 | **Language:** JAVA

Description: Vehicle data access object managing fleet queries, unit insertion, and status updates.

```
package com.rental.dao;

import com.rental.model.Vehicle;
import com.rental.util.DBConnection;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class VehicleDAO {

    public boolean addVehicle(Vehicle vehicle) {
        String sql="INSERT INTO vehicles(vehicle_number,brand,model,type,price_per_day,statu
          ■ s) VALUES(?,?,?,?,?,?)";

        try(Connection con=DBConnection.getConnection()) {
            if(con==null)return false;

            try(PreparedStatement ps=con.prepareStatement(sql)) {
                ps.setString(1,vehicle.getVehicleNumber());
                ps.setString(2,vehicle.getBrand());
                ps.setString(3,vehicle.getModel());
                ps.setString(4,vehicle.getType());
                ps.setDouble(5,vehicle.getPricePerDay());
                ps.setString(6,"AVAILABLE");
                return ps.executeUpdate()>0;
```

```
            }
        } catch(SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public boolean updateVehicle(Vehicle vehicle) {
        String sql="UPDATE vehicles SET vehicle_number=?,brand=?,model=?,type=?,price_per_da
          ■ y=?,status=? WHERE id=?";

        try(Connection con=DBConnection.getConnection()) {
            if(con==null)return false;

            try(PreparedStatement ps=con.prepareStatement(sql)) {
                ps.setString(1,vehicle.getVehicleNumber());
                ps.setString(2,vehicle.getBrand());
                ps.setString(3,vehicle.getModel());
                ps.setString(4,vehicle.getType());
                ps.setDouble(5,vehicle.getPricePerDay());
                ps.setString(6,vehicle.getStatus());
                ps.setInt(7,vehicle.getId());
                return ps.executeUpdate()>0;
            }
        } catch(SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public boolean deleteVehicle(int id) {
        String sql="DELETE FROM vehicles WHERE id=?";

        try(Connection con=DBConnection.getConnection()) {
            if(con==null)return false;

            try(PreparedStatement ps=con.prepareStatement(sql)) {
                ps.setInt(1,id);
                return ps.executeUpdate()>0;
            }
        } catch(SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public Vehicle getVehicleById(int id) {
        String sql="SELECT * FROM vehicles WHERE id=?";

        try(Connection con=DBConnection.getConnection()) {
            if(con==null)return null;

            try(PreparedStatement ps=con.prepareStatement(sql)) {
                ps.setInt(1,id);

                try(ResultSet rs=ps.executeQuery()) {
                    if(rs.next())return mapVehicle(rs);
                }
            }
        } catch(SQLException e) {
            e.printStackTrace();
        }
        return null;
    }

    public List<Vehicle> getAllVehicles() {
        List<Vehicle> vehicles=new ArrayList<>();
        String sql="SELECT * FROM vehicles ORDER BY id DESC";

        try(Connection con=DBConnection.getConnection()) {
            if(con==null)return vehicles;

            try(PreparedStatement ps=con.prepareStatement(sql);
                ResultSet rs=ps.executeQuery()) {

                while(rs.next())vehicles.add(mapVehicle(rs));
            }
        } catch(SQLException e) {
            e.printStackTrace();
        }
        return vehicles;
    }

    public List<Vehicle> getAvailableVehicles() {
        List<Vehicle> vehicles=new ArrayList<>();
        String sql="SELECT * FROM vehicles WHERE status='AVAILABLE' ORDER BY id DESC";
```

```
        try(Connection con=DBConnection.getConnection()) {
            if(con==null)return vehicles;

            try(PreparedStatement ps=con.prepareStatement(sql);
                ResultSet rs=ps.executeQuery()) {

                while(rs.next())vehicles.add(mapVehicle(rs));
            }
        } catch(SQLException e) {
            e.printStackTrace();
        }
        return vehicles;
    }

    public int countTotalVehicles() {
        String sql = "SELECT COUNT(*) FROM vehicles";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return rs.getInt(1);
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return 0;
    }

    public int countAvailableVehicles() {
        String sql = "SELECT COUNT(*) FROM vehicles WHERE status='AVAILABLE'";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return rs.getInt(1);
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return 0;
    }

    private Vehicle mapVehicle(ResultSet rs) throws SQLException {
        Vehicle vehicle=new Vehicle();
        vehicle.setId(rs.getInt("id"));
        vehicle.setVehicleNumber(rs.getString("vehicle_number"));
        vehicle.setBrand(rs.getString("brand"));
        vehicle.setModel(rs.getString("model"));
        vehicle.setType(rs.getString("type"));
        vehicle.setPricePerDay(rs.getDouble("price_per_day"));
        vehicle.setStatus(rs.getString("status"));
        return vehicle;
    }
}
```

File: src/main/java/com/rental/dao/BookingDAO.java | **Lines:** 273 | **Language:** JAVA

Description: Booking DAO featuring transaction row-level locking, date overlap validation, and customer history.

```
package com.rental.dao;

import com.rental.model.Booking;
import com.rental.util.DBConnection;
import java.sql.*;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;

public class BookingDAO {

    public boolean createBooking(Booking booking) {
        if (booking.getStartDate() == null || booking.getEndDate() == null) return false;
        if (!booking.getEndDate().after(booking.getStartDate())) return false;

        String vehicleSql = "SELECT price_per_day, status FROM vehicles WHERE id=? FOR UPDAT
          ■ E";
        String overlapSql = "SELECT id FROM bookings WHERE vehicle_id=? AND status IN ('PEND
          ■ ING','CONFIRMED') AND start_date < ? AND end_date > ? LIMIT 1";
        String insertSql = "INSERT INTO bookings(user_id, vehicle_id, start_date, end_date,
          ■ total_amount, status) VALUES(?,?,?,?,?,'PENDING')";

        try (Connection con = DBConnection.getConnection()) {
            if (con == null) return false;
            con.setAutoCommit(false);

            try {
                double pricePerDay;
```

```
            try (PreparedStatement ps = con.prepareStatement(vehicleSql)) {
                ps.setInt(1, booking.getVehicleId());

                try (ResultSet rs = ps.executeQuery()) {
                    if (!rs.next() || !"AVAILABLE".equalsIgnoreCase(rs.getString("status
                      ■ "))) {
                        con.rollback();
                        return false;
                    }
                    pricePerDay = rs.getDouble("price_per_day");
                }
            }

            try (PreparedStatement ps = con.prepareStatement(overlapSql)) {
                ps.setInt(1, booking.getVehicleId());
                ps.setDate(2, booking.getEndDate());
                ps.setDate(3, booking.getStartDate());

                try (ResultSet rs = ps.executeQuery()) {
                    if (rs.next()) {
                        con.rollback();
                        return false;
                    }
                }
            }

            long days = ChronoUnit.DAYS.between(
                booking.getStartDate().toLocalDate(),
                booking.getEndDate().toLocalDate()
            );
            if (days <= 0) days = 1;

            double totalAmount = days * pricePerDay;

            try (PreparedStatement ps = con.prepareStatement(insertSql, Statement.RETURN
              ■ _GENERATED_KEYS)) {
                ps.setInt(1, booking.getUserId());
                ps.setInt(2, booking.getVehicleId());
                ps.setDate(3, booking.getStartDate());
                ps.setDate(4, booking.getEndDate());
                ps.setDouble(5, totalAmount);

                if (ps.executeUpdate() == 0) {
                    con.rollback();
                    return false;
                }

                try (ResultSet keys = ps.getGeneratedKeys()) {
                    if (keys.next()) booking.setId(keys.getInt(1));
                }
            }

            booking.setTotalAmount(totalAmount);
            booking.setStatus("PENDING");
            con.commit();
            return true;

        } catch (SQLException e) {
            con.rollback();
            throw e;
        } finally {
            con.setAutoCommit(true);
        }

    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public List<Booking> getBookingsByUser(int userId) {
    List<Booking> bookings = new ArrayList<>();
    String sql = "SELECT b.*, v.brand, v.model, v.vehicle_number " +
                 "FROM bookings b " +
                 "LEFT JOIN vehicles v ON b.vehicle_id = v.id " +
                 "WHERE b.user_id=? ORDER BY b.id DESC";

    try (Connection con = DBConnection.getConnection()) {
        if (con == null) return bookings;

        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, userId);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    Booking b = mapBooking(rs);
```

```
                    String brand = rs.getString("brand");
                    String model = rs.getString("model");
                    b.setVehicleName((brand != null ? brand : "") + " " + (model != null
                      ■  ? model : ""));
                    b.setVehicleNumber(rs.getString("vehicle_number"));
                    bookings.add(b);
                }
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return bookings;
}

public List<Booking> getAllBookings() {
    List<Booking> bookings = new ArrayList<>();
    String sql = "SELECT b.*, v.brand, v.model, v.vehicle_number, u.name AS user_name, u
      ■ .email AS user_email " +
                 "FROM bookings b " +
                 "LEFT JOIN vehicles v ON b.vehicle_id = v.id " +
                 "LEFT JOIN users u ON b.user_id = u.id " +
                 "ORDER BY b.id DESC";

    try (Connection con = DBConnection.getConnection()) {
        if (con == null) return bookings;

        try (PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                Booking b = mapBooking(rs);
                String brand = rs.getString("brand");
                String model = rs.getString("model");
                b.setVehicleName((brand != null ? brand : "") + " " + (model != null ? m
                  ■ odel : ""));
                b.setVehicleNumber(rs.getString("vehicle_number"));
                b.setUserName(rs.getString("user_name"));
                b.setUserEmail(rs.getString("user_email"));
                bookings.add(b);
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return bookings;
}

public Booking getBookingById(int id) {
    String sql = "SELECT b.*, v.brand, v.model, v.vehicle_number, u.name AS user_name, u
      ■ .email AS user_email " +
                 "FROM bookings b " +
                 "LEFT JOIN vehicles v ON b.vehicle_id = v.id " +
                 "LEFT JOIN users u ON b.user_id = u.id " +
                 "WHERE b.id=?";

    try (Connection con = DBConnection.getConnection()) {
        if (con == null) return null;

        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    Booking b = mapBooking(rs);
                    String brand = rs.getString("brand");
                    String model = rs.getString("model");
                    b.setVehicleName((brand != null ? brand : "") + " " + (model != null
                      ■  ? model : ""));
                    b.setVehicleNumber(rs.getString("vehicle_number"));
                    b.setUserName(rs.getString("user_name"));
                    b.setUserEmail(rs.getString("user_email"));
                    return b;
                }
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return null;
}

public boolean cancelBooking(int bookingId, int userId) {
    String sql = "UPDATE bookings SET status='CANCELLED' WHERE id=? AND user_id=? AND st
      ■ atus IN ('PENDING', 'CONFIRMED')";
    try (Connection con = DBConnection.getConnection()) {
```

```
            if (con == null) return false;

            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setInt(1, bookingId);
                ps.setInt(2, userId);
                return ps.executeUpdate() > 0;
            }
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public boolean updateBookingStatus(int bookingId, String status) {
        if (!"CONFIRMED".equals(status) && !"REJECTED".equals(status) && !"CANCELLED".equals
          ■ (status)) {
            return false;
        }

        String sql = "UPDATE bookings SET status=? WHERE id=?";

        try (Connection con = DBConnection.getConnection()) {
            if (con == null) return false;

            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setString(1, status);
                ps.setInt(2, bookingId);
                return ps.executeUpdate() > 0;
            }
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public int countTotalBookings() {
        String sql = "SELECT COUNT(*) FROM bookings";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return rs.getInt(1);
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return 0;
    }

    public int countPendingBookings() {
        String sql = "SELECT COUNT(*) FROM bookings WHERE status='PENDING'";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return rs.getInt(1);
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return 0;
    }

    public double calculateTotalRevenue() {
        String sql = "SELECT COALESCE(SUM(amount), 0) FROM payments WHERE payment_status='SU
          ■ CCESS'";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return rs.getDouble(1);
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return 0.0;
    }

    private Booking mapBooking(ResultSet rs) throws SQLException {
        Booking booking = new Booking();
        booking.setId(rs.getInt("id"));
        booking.setUserId(rs.getInt("user_id"));
        booking.setVehicleId(rs.getInt("vehicle_id"));
        booking.setStartDate(rs.getDate("start_date"));
        booking.setEndDate(rs.getDate("end_date"));
        booking.setTotalAmount(rs.getDouble("total_amount"));
        booking.setStatus(rs.getString("status"));
        return booking;
    }
}
```

File: src/main/java/com/rental/dao/PaymentDAO.java	**Lines:** 108 | **Language:** JAVA
Description: Payment DAO processing simulated payment persistence and revenue aggregation.	

```
package com.rental.dao;

import com.rental.model.Payment;
import com.rental.util.DBConnection;
import java.sql.*;

public class PaymentDAO {

    public boolean processMockPayment(Payment payment) {
        String bookingSql = "SELECT total_amount, status FROM bookings WHERE id=? FOR UPDATE
          ■ ";
        String insertSql = "INSERT INTO payments(booking_id, amount, payment_status) VALUES(
          ■ ?,?,?)";
        String confirmSql = "UPDATE bookings SET status='CONFIRMED' WHERE id=? AND status='P
          ■ ENDING'";

        String paymentStatus = payment.getPaymentStatus();

        if (!"SUCCESS".equals(paymentStatus) && !"FAILED".equals(paymentStatus)) {
            return false;
        }

        try (Connection con = DBConnection.getConnection()) {
            if (con == null) return false;
            con.setAutoCommit(false);

            try {
                double amount;

                try (PreparedStatement ps = con.prepareStatement(bookingSql)) {
                    ps.setInt(1, payment.getBookingId());

                    try (ResultSet rs = ps.executeQuery()) {
                        if (!rs.next() || !"PENDING".equals(rs.getString("status"))) {
                            con.rollback();
                            return false;
                        }
                        amount = rs.getDouble("total_amount");
                    }
                }

                try (PreparedStatement ps = con.prepareStatement(insertSql, Statement.RETURN
                  ■ _GENERATED_KEYS)) {
                    ps.setInt(1, payment.getBookingId());
                    ps.setDouble(2, amount);
                    ps.setString(3, paymentStatus);

                    if (ps.executeUpdate() == 0) {
                        con.rollback();
                        return false;
                    }

                    try (ResultSet keys = ps.getGeneratedKeys()) {
                        if (keys.next()) payment.setId(keys.getInt(1));
                    }
                }

                if ("SUCCESS".equals(paymentStatus)) {
                    try (PreparedStatement ps = con.prepareStatement(confirmSql)) {
                        ps.setInt(1, payment.getBookingId());

                        if (ps.executeUpdate() != 1) {
                            con.rollback();
                            return false;
                        }
                    }
                }

                payment.setAmount(amount);
                con.commit();
                return true;

            } catch (SQLException e) {
                con.rollback();
                throw e;
            } finally {
                con.setAutoCommit(true);
            }

        } catch (SQLException e) {
            e.printStackTrace();
```

```
            return false;
        }
    }

    public Payment getPaymentByBookingId(int bookingId) {
        String sql = "SELECT * FROM payments WHERE booking_id=? ORDER BY id DESC LIMIT 1";

        try (Connection con = DBConnection.getConnection()) {
            if (con == null) return null;

            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setInt(1, bookingId);

                try (ResultSet rs = ps.executeQuery()) {
                    if (rs.next()) {
                        Payment payment = new Payment();
                        payment.setId(rs.getInt("id"));
                        payment.setBookingId(rs.getInt("booking_id"));
                        payment.setAmount(rs.getDouble("amount"));
                        payment.setPaymentStatus(rs.getString("payment_status"));
                        payment.setPaymentDate(rs.getTimestamp("payment_date"));
                        return payment;
                    }
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

D.3 Database Connection & Utility Classes

Connection factory and database connectivity verification harness

File: src/main/java/com/rental/util/DBConnection.java | **Lines:** 21 | **Language:** JAVA

Description: Thread-safe JDBC connection provider using DriverManager. (Password credential redacted for security).

```
package com.rental.util;

import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {

    private static final String URL="jdbc:mysql://localhost:3306/rental_vehicle";
    private static final String USER="root";
    private static final String PASSWORD = "••••••••••••" /* [REDACTED_SECURE_PASSWORD] */;

    public static Connection getConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(URL,USER,PASSWORD);
        } catch(Exception e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

File: src/main/java/com/rental/util/TESTConnection.java | **Lines:** 17 | **Language:** JAVA

Description: Standalone console verification utility to test active database connectivity.

```
package com.rental.util;

import java.sql.Connection;

public class TESTConnection {

    public static void main(String[] args) {

        Connection connection=DBConnection.getConnection();

        if(connection!=null) {
            System.out.println("Database connected successfully");
        } else {
```

```
            System.out.println("Database connection failed");
        }
    }
}
```

D.4 Public & Customer Servlets / Controllers

HTTP Controllers handling routing, input validation, and user sessions

File: src/main/java/com/rental/controller/VehicleServlet.java | **Lines:** 28 | **Language:** JAVA

Description: Public catalog servlet loading available vehicles for guest and user browsing.

```
package com.rental.controller;

import com.rental.dao.VehicleDAO;
import com.rental.model.Vehicle;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;
import java.util.List;

@WebServlet("/vehicles")
public class VehicleServlet extends HttpServlet {
    private final VehicleDAO vehicleDAO=new VehicleDAO();

    @Override
    protected void doGet(HttpServletRequest request,HttpServletResponse response)
            throws ServletException,IOException {

        List<Vehicle> vehicles=vehicleDAO.getAvailableVehicles();

        request.setAttribute("vehicles",vehicles);
        request.getRequestDispatcher("/vehicles.jsp").forward(request,response);
    }
}
```

File: src/main/java/com/rental/controller/VehicleDetailsServlet.java | **Lines:** 39 | **Language:** JAVA

Description: Details servlet rendering complete vehicle specifications and rental parameters.

```
package com.rental.controller;

import com.rental.dao.VehicleDAO;
import com.rental.model.Vehicle;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet("/vehicle-details")
public class VehicleDetailsServlet extends HttpServlet {
    private final VehicleDAO vehicleDAO=new VehicleDAO();

    @Override
    protected void doGet(HttpServletRequest request,HttpServletResponse response)
            throws ServletException,IOException {

        String id=request.getParameter("id");

        try{
            int vehicleId=Integer.parseInt(id);
            Vehicle vehicle=vehicleDAO.getVehicleById(vehicleId);

            if(vehicle==null){
                response.sendError(HttpServletResponse.SC_NOT_FOUND,"Vehicle not found");
                return;
            }

            request.setAttribute("vehicle",vehicle);
            request.getRequestDispatcher("/vehicle-details.jsp").forward(request,response);

        }catch(NumberFormatException e){
            response.sendError(HttpServletResponse.SC_BAD_REQUEST,"Invalid vehicle ID");
```

```
        }
    }
}
```

File: src/main/java/com/rental/controller/RegisterServlet.java | **Lines:** 58 | **Language:** JAVA

Description: Customer registration controller with BCrypt password hashing and input checks.

```
package com.rental.controller;

import com.rental.dao.UserDAO;
import com.rental.model.User;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import org.mindrot.jbcrypt.BCrypt;
import java.io.IOException;

@WebServlet("/register")
public class RegisterServlet extends HttpServlet {

    private final UserDAO userDAO=new UserDAO();

    @Override
    protected void doPost(HttpServletRequest request,HttpServletResponse response)
            throws ServletException,IOException {

        request.setCharacterEncoding("UTF-8");

        String name=request.getParameter("name");
        String email=request.getParameter("email");
        String password=request.getParameter("password");

        if(name==null||email==null||password==null||
           name.isBlank()||email.isBlank()||password.length()<8) {
            request.setAttribute("error","Enter valid details. Password must contain at leas
              ■ t 8 characters.");
            request.getRequestDispatcher("/register.jsp").forward(request,response);
            return;
        }

        email=email.trim().toLowerCase();

        if(userDAO.getUserByEmail(email)!=null) {
            request.setAttribute("error","Email is already registered.");
            request.getRequestDispatcher("/register.jsp").forward(request,response);
            return;
        }

        User user=new User();
        user.setName(name.trim());
        user.setEmail(email);
        user.setPassword(BCrypt.hashpw(password,BCrypt.gensalt()));
        user.setRole("USER");

        if(userDAO.registerUser(user)) {
            response.sendRedirect(request.getContextPath()+"/login.jsp?registered=1");
        } else {
            request.setAttribute("error","Registration failed. Please try again.");
            request.getRequestDispatcher("/register.jsp").forward(request,response);
        }
    }
}
```

File: src/main/java/com/rental/controller/LoginServlet.java | **Lines:** 60 | **Language:** JAVA

Description: Authentication controller verifying BCrypt credentials and setting session state.

```
package com.rental.controller;

import com.rental.dao.UserDAO;
import com.rental.model.User;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import org.mindrot.jbcrypt.BCrypt;
import java.io.IOException;
```

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    private final UserDAO userDAO=new UserDAO();

    @Override
    protected void doPost(HttpServletRequest request,HttpServletResponse response)
            throws ServletException,IOException {

        request.setCharacterEncoding("UTF-8");

        String email=request.getParameter("email");
        String password=request.getParameter("password");

        if(email==null||password==null||email.isBlank()||password.isEmpty()){
            request.setAttribute("error","Enter your email and password.");
            request.getRequestDispatcher("/login.jsp").forward(request,response);
            return;
        }

        User user=userDAO.getUserByEmail(email.trim().toLowerCase());

        if(user==null||user.getPassword()==null||
                !BCrypt.checkpw(password,user.getPassword())){
            request.setAttribute("error","Invalid email or password.");
            request.getRequestDispatcher("/login.jsp").forward(request,response);
            return;
        }

        HttpSession oldSession=request.getSession(false);
        if(oldSession!=null){
            oldSession.invalidate();
        }

        HttpSession session=request.getSession(true);
        session.setAttribute("userId",user.getId());
        session.setAttribute("userName",user.getName());
        session.setAttribute("userRole",user.getRole());
        session.setMaxInactiveInterval(30*60);

        if("ADMIN".equalsIgnoreCase(user.getRole())){
            response.sendRedirect(request.getContextPath()+"/admin/dashboard");
        }else{
            response.sendRedirect(request.getContextPath()+"/vehicles");
        }
    }
}
```

File: src/main/java/com/rental/controller/BookingServlet.java | **Lines:** 134 | **Language:** JAVA

Description: Reservation controller validating date ranges and preventing overlap collisions.

```
package com.rental.controller;

import com.rental.dao.BookingDAO;
import com.rental.dao.VehicleDAO;
import com.rental.model.Booking;
import com.rental.model.Vehicle;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;
import java.sql.Date;
import java.time.LocalDate;

@WebServlet("/booking")
public class BookingServlet extends HttpServlet {

    private final BookingDAO bookingDAO = new BookingDAO();
    private final VehicleDAO vehicleDAO = new VehicleDAO();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp?error=" +
                    java.net.URLEncoder.encode("Please log in to book a vehicle.", "UTF-8"))
                      ■ ;
            return;
        }
```

```
        String vehicleIdStr = request.getParameter("vehicleId");
        if (vehicleIdStr == null || vehicleIdStr.isBlank()) {
            response.sendRedirect(request.getContextPath() + "/vehicles");
            return;
        }

        try {
            int vehicleId = Integer.parseInt(vehicleIdStr.trim());
            Vehicle vehicle = vehicleDAO.getVehicleById(vehicleId);

            if (vehicle == null) {
                response.sendError(HttpServletResponse.SC_NOT_FOUND, "Vehicle not found");
                return;
            }

            request.setAttribute("vehicle", vehicle);
            request.getRequestDispatcher("/booking.jsp").forward(request, response);

        } catch (NumberFormatException e) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid vehicle ID");
        }
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");
        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        int userId = (Integer) session.getAttribute("userId");
        String vehicleIdStr = request.getParameter("vehicleId");
        String startDateStr = request.getParameter("startDate");
        String endDateStr = request.getParameter("endDate");

        if (vehicleIdStr == null || startDateStr == null || endDateStr == null ||
            vehicleIdStr.isBlank() || startDateStr.isBlank() || endDateStr.isBlank()) {
            request.setAttribute("error", "All fields are required.");
            redispatchBookingForm(request, response, vehicleIdStr);
            return;
        }

        int vehicleId;
        Date startDate;
        Date endDate;

        try {
            vehicleId = Integer.parseInt(vehicleIdStr.trim());
            startDate = Date.valueOf(startDateStr.trim());
            endDate = Date.valueOf(endDateStr.trim());
        } catch (IllegalArgumentException e) {
            request.setAttribute("error", "Invalid date or vehicle selection format.");
            redispatchBookingForm(request, response, vehicleIdStr);
            return;
        }

        LocalDate today = LocalDate.now();
        if (startDate.toLocalDate().isBefore(today)) {
            request.setAttribute("error", "Start date cannot be in the past.");
            redispatchBookingForm(request, response, vehicleIdStr);
            return;
        }

        if (!endDate.after(startDate)) {
            request.setAttribute("error", "End date must be strictly after start date.");
            redispatchBookingForm(request, response, vehicleIdStr);
            return;
        }

        Booking booking = new Booking();
        booking.setUserId(userId);
        booking.setVehicleId(vehicleId);
        booking.setStartDate(startDate);
        booking.setEndDate(endDate);

        boolean created = bookingDAO.createBooking(booking);

        if (created) {
            // Forward to mock payment step
            response.sendRedirect(request.getContextPath() + "/payment?bookingId=" + booking
              ■ .getId());
        } else {
```

```
            request.setAttribute("error", "Vehicle is unavailable or has an overlapping book
              ■ ing for the selected dates.");
            redispatchBookingForm(request, response, vehicleIdStr);
        }
    }

    private void redispatchBookingForm(HttpServletRequest request, HttpServletResponse respo
      ■ nse, String vehicleIdStr)
            throws ServletException, IOException {
        try {
            int vehicleId = Integer.parseInt(vehicleIdStr.trim());
            Vehicle vehicle = vehicleDAO.getVehicleById(vehicleId);
            request.setAttribute("vehicle", vehicle);
        } catch (Exception ignored) {
        }
        request.getRequestDispatcher("/booking.jsp").forward(request, response);
    }
}
```

File: src/main/java/com/rental/controller/PaymentServlet.java | **Lines:** 135 | **Language:** JAVA

Description: Payment sandbox controller processing simulated card, UPI, and netbanking transactions.

```
package com.rental.controller;

import com.rental.dao.BookingDAO;
import com.rental.dao.PaymentDAO;
import com.rental.model.Booking;
import com.rental.model.Payment;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;

@WebServlet("/payment")
public class PaymentServlet extends HttpServlet {

    private final BookingDAO bookingDAO = new BookingDAO();
    private final PaymentDAO paymentDAO = new PaymentDAO();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        int userId = (Integer) session.getAttribute("userId");
        String role = (String) session.getAttribute("userRole");
        String bookingIdStr = request.getParameter("bookingId");

        if (bookingIdStr == null || bookingIdStr.isBlank()) {
            response.sendRedirect(request.getContextPath() + "/my-bookings");
            return;
        }

        try {
            int bookingId = Integer.parseInt(bookingIdStr.trim());
            Booking booking = bookingDAO.getBookingById(bookingId);

            if (booking == null) {
                response.sendError(HttpServletResponse.SC_NOT_FOUND, "Booking not found");
                return;
            }

            // Access control: only owner or admin can access payment
            if (booking.getUserId() != userId && !"ADMIN".equalsIgnoreCase(role)) {
                response.sendError(HttpServletResponse.SC_FORBIDDEN, "Access denied");
                return;
            }

            if ("CONFIRMED".equalsIgnoreCase(booking.getStatus())) {
                response.sendRedirect(request.getContextPath() + "/my-bookings?info=already_
                  ■ confirmed");
                return;
            }

            if ("CANCELLED".equalsIgnoreCase(booking.getStatus()) || "REJECTED".equalsIgnore
```

```
            ■ Case(booking.getStatus())) {
              response.sendRedirect(request.getContextPath() + "/my-bookings?error=invalid
                ■ _status");
              return;
            }

            request.setAttribute("booking", booking);
            request.getRequestDispatcher("/payment.jsp").forward(request, response);

        } catch (NumberFormatException e) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid booking ID");
        }
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");
        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        int userId = (Integer) session.getAttribute("userId");
        String role = (String) session.getAttribute("userRole");
        String bookingIdStr = request.getParameter("bookingId");
        String actionStatus = request.getParameter("paymentStatus"); // "SUCCESS" or "FAILED
          ■ "

        if (bookingIdStr == null || bookingIdStr.isBlank()) {
            response.sendRedirect(request.getContextPath() + "/my-bookings");
            return;
        }

        try {
            int bookingId = Integer.parseInt(bookingIdStr.trim());
            Booking booking = bookingDAO.getBookingById(bookingId);

            if (booking == null) {
                response.sendError(HttpServletResponse.SC_NOT_FOUND, "Booking not found");
                return;
            }

            if (booking.getUserId() != userId && !"ADMIN".equalsIgnoreCase(role)) {
                response.sendError(HttpServletResponse.SC_FORBIDDEN, "Access denied");
                return;
            }

            // Prevent duplicate payment if already confirmed
            if ("CONFIRMED".equalsIgnoreCase(booking.getStatus())) {
                response.sendRedirect(request.getContextPath() + "/my-bookings?info=already_
                  ■ confirmed");
                return;
            }

            String paymentStatus = ("FAILED".equalsIgnoreCase(actionStatus)) ? "FAILED" : "S
              ■ UCCESS";

            Payment payment = new Payment();
            payment.setBookingId(bookingId);
            payment.setAmount(booking.getTotalAmount());
            payment.setPaymentStatus(paymentStatus);

            boolean processed = paymentDAO.processMockPayment(payment);

            if (processed && "SUCCESS".equals(paymentStatus)) {
                response.sendRedirect(request.getContextPath() + "/my-bookings?payment=succe
                  ■ ss");
            } else if (processed && "FAILED".equals(paymentStatus)) {
                response.sendRedirect(request.getContextPath() + "/payment?bookingId=" + boo
                  ■ kingId + "&error=Mock+payment+failed.+Please+retry.");
            } else {
                response.sendRedirect(request.getContextPath() + "/payment?bookingId=" + boo
                  ■ kingId + "&error=Unable+to+process+payment.+Please+try+again.");
            }

        } catch (NumberFormatException e) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid booking ID");
        }
    }
}
```

File: src/main/java/com/rental/controller/MyBookingsServlet.java | **Lines:** 35 | **Language:** JAVA

Description: Customer reservations controller rendering personal booking history and statuses.

```
package com.rental.controller;

import com.rental.dao.BookingDAO;
import com.rental.model.Booking;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;
import java.util.List;

@WebServlet("/my-bookings")
public class MyBookingsServlet extends HttpServlet {

    private final BookingDAO bookingDAO = new BookingDAO();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        int userId = (Integer) session.getAttribute("userId");
        List<Booking> bookings = bookingDAO.getBookingsByUser(userId);

        request.setAttribute("bookings", bookings);
        request.getRequestDispatcher("/my-bookings.jsp").forward(request, response);
    }
}
```

File: src/main/java/com/rental/controller/CancelBookingServlet.java | **Lines:** 50 | **Language:** JAVA

Description: Reservation cancellation controller releasing vehicle inventory.

```
package com.rental.controller;

import com.rental.dao.BookingDAO;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;

@WebServlet("/cancel-booking")
public class CancelBookingServlet extends HttpServlet {

    private final BookingDAO bookingDAO = new BookingDAO();

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        int userId = (Integer) session.getAttribute("userId");
        String bookingIdStr = request.getParameter("bookingId");

        if (bookingIdStr != null && !bookingIdStr.isBlank()) {
            try {
                int bookingId = Integer.parseInt(bookingIdStr.trim());
                boolean cancelled = bookingDAO.cancelBooking(bookingId, userId);
                if (cancelled) {
                    response.sendRedirect(request.getContextPath() + "/my-bookings?msg=cance
                      ■ lled");
                    return;
                }
            } catch (NumberFormatException ignored) {
            }
```

```
        }

        response.sendRedirect(request.getContextPath() + "/my-bookings?error=cancel_failed")
          ■ ;
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        doPost(request, response);
    }
}
```

File: src/main/java/com/rental/controller/LogoutServlet.java | **Lines:** 31 | **Language:** JAVA

Description: Session termination controller invalidating session state and cookies.

```
package com.rental.controller;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;

@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate();
        }

        response.sendRedirect(request.getContextPath() + "/login.jsp?logout=1");
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        doGet(request, response);
    }
}
```

D.5 Administrator Portal Servlets

Administrative controllers for operational metrics, fleet, and reservations

File: src/main/java/com/rental/controller/AdminDashboardServlet.java | **Lines:** 51 | **Language:** JAVA

Description: Admin dashboard controller computing gross revenue, fleet size, and active bookings.

```
package com.rental.controller;

import com.rental.dao.BookingDAO;
import com.rental.dao.VehicleDAO;
import com.rental.model.Booking;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;
import java.util.List;

@WebServlet("/admin/dashboard")
public class AdminDashboardServlet extends HttpServlet {

    private final VehicleDAO vehicleDAO = new VehicleDAO();
    private final BookingDAO bookingDAO = new BookingDAO();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
```

```
        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null ||
            !"ADMIN".equalsIgnoreCase((String) session.getAttribute("userRole"))) {
            response.sendRedirect(request.getContextPath() + "/login.jsp?error=" +
                    java.net.URLEncoder.encode("Admin access required.", "UTF-8"));
            return;
        }

        int totalVehicles = vehicleDAO.countTotalVehicles();
        int availableVehicles = vehicleDAO.countAvailableVehicles();
        int totalBookings = bookingDAO.countTotalBookings();
        int pendingBookings = bookingDAO.countPendingBookings();
        double totalRevenue = bookingDAO.calculateTotalRevenue();

        List<Booking> allBookings = bookingDAO.getAllBookings();
        List<Booking> recentBookings = allBookings.size() > 5 ? allBookings.subList(0, 5) :
          ■ allBookings;

        request.setAttribute("totalVehicles", totalVehicles);
        request.setAttribute("availableVehicles", availableVehicles);
        request.setAttribute("totalBookings", totalBookings);
        request.setAttribute("pendingBookings", pendingBookings);
        request.setAttribute("totalRevenue", totalRevenue);
        request.setAttribute("recentBookings", recentBookings);

        request.getRequestDispatcher("/admin/dashboard.jsp").forward(request, response);
    }
}
```

File: src/main/java/com/rental/controller/AdminVehicleServlet.java | **Lines:** 135 | **Language:** JAVA

Description: Fleet management controller handling vehicle additions, status changes, and deletions.

```
package com.rental.controller;

import com.rental.dao.VehicleDAO;
import com.rental.model.Vehicle;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;
import java.util.List;

@WebServlet("/admin/vehicles")
public class AdminVehicleServlet extends HttpServlet {

    private final VehicleDAO vehicleDAO = new VehicleDAO();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || !"ADMIN".equalsIgnoreCase((String) session.getAttribute("user
          ■ Role"))) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        List<Vehicle> vehicles = vehicleDAO.getAllVehicles();
        request.setAttribute("vehicles", vehicles);
        request.getRequestDispatcher("/admin/vehicles.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");
        HttpSession session = request.getSession(false);
        if (session == null || !"ADMIN".equalsIgnoreCase((String) session.getAttribute("user
          ■ Role"))) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        String action = request.getParameter("action");
        if (action == null) action = "";

        switch (action) {
            case "add": {
```

```
            String vehicleNumber = request.getParameter("vehicleNumber");
            String brand = request.getParameter("brand");
            String model = request.getParameter("model");
            String type = request.getParameter("type");
            String priceStr = request.getParameter("pricePerDay");

            if (vehicleNumber != null && brand != null && model != null && type != null
              ■ && priceStr != null) {
                try {
                    double pricePerDay = Double.parseDouble(priceStr.trim());
                    Vehicle vehicle = new Vehicle();
                    vehicle.setVehicleNumber(vehicleNumber.trim());
                    vehicle.setBrand(brand.trim());
                    vehicle.setModel(model.trim());
                    vehicle.setType(type.trim());
                    vehicle.setPricePerDay(pricePerDay);
                    vehicle.setStatus("AVAILABLE");

                    vehicleDAO.addVehicle(vehicle);
                } catch (NumberFormatException ignored) {
                }
            }
            break;
        }

        case "update": {
            String idStr = request.getParameter("id");
            String vehicleNumber = request.getParameter("vehicleNumber");
            String brand = request.getParameter("brand");
            String model = request.getParameter("model");
            String type = request.getParameter("type");
            String priceStr = request.getParameter("pricePerDay");
            String status = request.getParameter("status");

            if (idStr != null && vehicleNumber != null && brand != null && model != null
              ■  && priceStr != null) {
                try {
                    int id = Integer.parseInt(idStr.trim());
                    double pricePerDay = Double.parseDouble(priceStr.trim());
                    Vehicle vehicle = new Vehicle();
                    vehicle.setId(id);
                    vehicle.setVehicleNumber(vehicleNumber.trim());
                    vehicle.setBrand(brand.trim());
                    vehicle.setModel(model.trim());
                    vehicle.setType(type != null ? type.trim() : "Sedan");
                    vehicle.setPricePerDay(pricePerDay);
                    vehicle.setStatus(status != null ? status.trim() : "AVAILABLE");

                    vehicleDAO.updateVehicle(vehicle);
                } catch (NumberFormatException ignored) {
                }
            }
            break;
        }

        case "delete": {
            String idStr = request.getParameter("id");
            if (idStr != null) {
                try {
                    int id = Integer.parseInt(idStr.trim());
                    vehicleDAO.deleteVehicle(id);
                } catch (NumberFormatException ignored) {
                }
            }
            break;
        }

        case "toggleStatus": {
            String idStr = request.getParameter("id");
            if (idStr != null) {
                try {
                    int id = Integer.parseInt(idStr.trim());
                    Vehicle vehicle = vehicleDAO.getVehicleById(id);
                    if (vehicle != null) {
                        String newStatus = "AVAILABLE".equalsIgnoreCase(vehicle.getStatu
                          ■ s()) ? "UNAVAILABLE" : "AVAILABLE";
                        vehicle.setStatus(newStatus);
                        vehicleDAO.updateVehicle(vehicle);
                    }
                } catch (NumberFormatException ignored) {
                }
            }
            break;
        }
    }
    response.sendRedirect(request.getContextPath() + "/admin/vehicles");
```

```
    }
}
```

File: src/main/java/com/rental/controller/AdminBookingServlet.java | **Lines:** 58 | **Language:** JAVA

Description: Admin reservations controller providing centralized status override controls.

```
package com.rental.controller;

import com.rental.dao.BookingDAO;
import com.rental.model.Booking;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;
import java.util.List;

@WebServlet("/admin/bookings")
public class AdminBookingServlet extends HttpServlet {

    private final BookingDAO bookingDAO = new BookingDAO();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || !"ADMIN".equalsIgnoreCase((String) session.getAttribute("user
          ■ Role"))) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        List<Booking> bookings = bookingDAO.getAllBookings();
        request.setAttribute("bookings", bookings);
        request.getRequestDispatcher("/admin/bookings.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");
        HttpSession session = request.getSession(false);
        if (session == null || !"ADMIN".equalsIgnoreCase((String) session.getAttribute("user
          ■ Role"))) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        String bookingIdStr = request.getParameter("bookingId");
        String status = request.getParameter("status");

        if (bookingIdStr != null && status != null) {
            try {
                int bookingId = Integer.parseInt(bookingIdStr.trim());
                bookingDAO.updateBookingStatus(bookingId, status.trim().toUpperCase());
            } catch (NumberFormatException ignored) {
            }
        }

        response.sendRedirect(request.getContextPath() + "/admin/bookings");
    }
}
```

D.6 Presentation Views / JavaServer Pages (JSP)

Dynamic web presentation templates with modern responsive markup

File: src/main/webapp/index.jsp | **Lines:** 54 | **Language:** JSP

Description: Modern landing page with hero banner, feature highlights, and call-to-actions.

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>RentRide — Modern Vehicle Rental Experience</title>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body>
    <%@ include file="includes/navbar.jsp" %>

    <main class="main-content">
        <div class="landing-main">
            <section class="hero-section">
                <span class="badge">Drive Your Next Journey</span>
                <h1>Premium Vehicles,<br>Uncompromised Freedom.</h1>
                <p class="subtitle">Experience hassle-free car and motorcycle rentals at tra
                  ■ nsparent daily rates with instant online reservation and verified fleet
                  ■ quality.</p>

                <div class="hero-actions">
                    <a href="${pageContext.request.contextPath}/vehicles" class="btn btn-pri
                      ■ mary">Browse Available Vehicles</a>
                    <% if (session.getAttribute("userId") == null) { %>
                        <a href="${pageContext.request.contextPath}/register.jsp" class="btn
                          ■  btn-secondary">Create Free Account</a>
                    <% } else { %>
                        <a href="${pageContext.request.contextPath}/my-bookings" class="btn
                          ■ btn-secondary">View My Bookings</a>
                    <% } %>
                </div>
            </section>

            <section class="features-grid">
                <div class="feature-card">
                    <div class="feature-icon">■</div>
                    <h3>Instant Online Booking</h3>
                    <p>Reserve vehicles in seconds with live availability and automatic over
                      ■ lap scheduling protection.</p>
                </div>
                <div class="feature-card">
                    <div class="feature-icon">■■</div>
                    <h3>Verified & Inspected Fleet</h3>
                    <p>All cars and bikes are thoroughly cleaned, serviced, and insured befo
                      ■ re every single journey.</p>
                </div>
                <div class="feature-card">
                    <div class="feature-icon">■</div>
                    <h3>Transparent Pricing</h3>
                    <p>Clear daily pricing with zero hidden surcharges, fast mock sandbox pa
                      ■ yments, and instant confirmation.</p>
                </div>
            </section>
        </div>
    </main>

    <footer class="footer">
        <p>&copy; 2026 RentRide Inc. Developed with Java Servlets, JSP, JDBC, MySQL & Tomcat
          ■  9.</p>
    </footer>
</body>
</html>
```

File: src/main/webapp/vehicles.jsp	**Lines:** 76 | **Language:** JSP
Description: Fleet catalog page rendering vehicle photography cards and booking buttons.	

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="java.util.List,com.rental.model.Vehicle" %>
<%
    List<Vehicle> vehicles = (List<Vehicle>) request.getAttribute("vehicles");
    if (vehicles == null) {
        response.sendRedirect(request.getContextPath() + "/vehicles");
        return;
    }
%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,initial-scale=1">
    <title>Browse Vehicles | RentRide</title>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body>
    <%@ include file="includes/navbar.jsp" %>
    <main class="main-content">
```

```
        <div class="vehicles-main">
            <h1>Find Your Ideal Vehicle</h1>
            <p class="subtitle">Explore our maintained fleet ready for immediate booking.</p
              ■ >

            <div class="vehicle-grid">
                <%
                    if (vehicles != null && !vehicles.isEmpty()) {
                        for (Vehicle v : vehicles) {
                %>
                    <article class="vehicle-card">
                        <div class="vehicle-image-wrapper">
                            <img src="${pageContext.request.contextPath}/<%= v.getImageUrl()
                              ■  %>"
                                alt="<%= v.getBrand() %> <%= v.getModel() %>"
                                class="vehicle-photo"
                                loading="lazy"
                                onerror="this.src='${pageContext.request.contextPath}/image
                                  ■ s/vehicles/default.jpg'">
                        </div>

                        <div class="vehicle-info">
                            <span class="vehicle-type-tag"><%= v.getType() %></span>
                            <h2><%= v.getBrand() %> <%= v.getModel() %></h2>
                            <p class="vehicle-number">Reg: <%= v.getVehicleNumber() %></p>

                            <p class="vehicle-price">
                                ■<%= String.format("%.2f", v.getPricePerDay()) %>
                                <span>/ day</span>
                            </p>

                            <a class="btn btn-primary"
                               href="${pageContext.request.contextPath}/vehicle-details?id=<
                                 ■ %= v.getId() %>">
                                View Details & Book
                            </a>
                        </div>
                    </article>
                <%
                        }
                    } else {
                %>
                    <div class="empty-state" style="grid-column: 1/-1; text-align: center; p
                      ■ adding: 60px 20px; background: white; border-radius: 12px; border: 1
                      ■ px solid var(--border);">
                        <div style="font-size: 48px; margin-bottom: 12px;">■</div>
                        <h2>No Vehicles Currently Available</h2>
                        <p class="subtitle">All vehicles are currently booked. Please check
                          ■ back shortly.</p>
                    </div>
                <%
                    }
                %>
            </div>
        </div>
    </main>

    <footer class="footer">
        <p>&copy; 2026 RentRide Inc. Developed with Java Servlets, JSP, JDBC, MySQL & Tomcat
          ■  9.</p>
    </footer>
</body>
</html>
```

File: src/main/webapp/vehicle-details.jsp | **Lines:** 70 | **Language:** JSP

Description: Vehicle inspection page displaying full technical parameters and booking link.

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="com.rental.model.Vehicle" %>
<%
    Vehicle vehicle = (Vehicle) request.getAttribute("vehicle");
    if (vehicle == null) {
        response.sendRedirect(request.getContextPath() + "/vehicles");
        return;
    }
%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,initial-scale=1">
    <title><%= vehicle.getBrand() %> <%= vehicle.getModel() %> | RentRide</title>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
```

```
</head>
<body>
    <%@ include file="includes/navbar.jsp" %>

    <main class="main-content">
        <div class="details-container">
            <div class="details-card">
                <div class="details-image-wrapper">
                    <img src="${pageContext.request.contextPath}/<%= vehicle.getImageUrl() %
                      ■ >"
                         alt="<%= vehicle.getBrand() %> <%= vehicle.getModel() %>"
                         onerror="this.src='${pageContext.request.contextPath}/images/vehicl
                           ■ es/default.jpg'">
                </div>

                <div class="details-info">
                    <span class="vehicle-type-tag"><%= vehicle.getType() %></span>
                    <h1><%= vehicle.getBrand() %> <%= vehicle.getModel() %></h1>
                    <p class="vehicle-meta">Registration: <strong><%= vehicle.getVehicleNumb
                      ■ er() %></strong></p>
                    <p class="vehicle-meta">Status:
                        <span class="status-badge <%= "AVAILABLE".equalsIgnoreCase(vehicle.g
                          ■ etStatus()) ? "status-confirmed" : "status-cancelled" %>">
                            <%= vehicle.getStatus() %>
                        </span>
                    </p>

                    <div class="vehicle-price">
                        ■<%= String.format("%.2f", vehicle.getPricePerDay()) %>
                        <span>/ day</span>
                    </div>

                    <div style="margin-top: 15px; margin-bottom: 25px;">
                        <% if ("AVAILABLE".equalsIgnoreCase(vehicle.getStatus())) { %>
                            <a class="btn btn-primary" style="width: 100%;"
                               href="${pageContext.request.contextPath}/booking?vehicleId=<%
                                 ■ = vehicle.getId() %>">
                                Proceed to Booking
                            </a>
                        <% } else { %>
                            <div class="message error">This vehicle is currently unavailable
                              ■  for booking.</div>
                        <% } %>
                    </div>

                    <p>
                        <a href="${pageContext.request.contextPath}/vehicles" class="btn btn
                          ■ -outline" style="width: 100%;">
                            &larr; Back to Vehicle Catalog
                        </a>
                    </p>
                </div>
            </div>
        </div>
    </main>

    <footer class="footer">
        <p>&copy; 2026 RentRide Inc. Developed with Java Servlets, JSP, JDBC, MySQL & Tomcat
          ■  9.</p>
    </footer>
</body>
</html>
```

File: src/main/webapp/login.jsp	**Lines:** 61 | **Language:** JSP
Description: Unified authentication form supporting customer and administrator logins.	

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,initial-scale=1">
    <title>Login | RentRide</title>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body>
    <%@ include file="includes/navbar.jsp" %>

    <main class="main-content auth-wrapper">
        <div class="auth-container">
            <h2>Welcome Back</h2>
            <p class="subtitle">Log in to your RentRide account.</p>
            <% if (request.getAttribute("error") != null) { %>
```

```
                <div class="message error">
                    <%= request.getAttribute("error") %>
                </div>
            <% } %>

            <% if ("1".equals(request.getParameter("registered"))) { %>
                <div class="message success">
                    Registration successful! Please sign in with your credentials.
                </div>
            <% } %>

            <% if ("1".equals(request.getParameter("logout"))) { %>
                <div class="message info">
                    You have been logged out successfully.
                </div>
            <% } %>

            <form action="${pageContext.request.contextPath}/login" method="post">
                <div class="form-group">
                    <label for="email">Email Address</label>
                    <input type="email" id="email" name="email" placeholder="name@example.co
                      ■ m" required autocomplete="email">
                </div>

                <div class="form-group">
                    <label for="password">Password</label>
                    <input type="password" id="password" name="password" placeholder="••••••
                      ■ ••" required autocomplete="current-password">
                </div>

                <button type="submit" class="btn btn-primary" style="width:100%; margin-top:
                  ■ 10px;">Sign In</button>
            </form>

            <p class="auth-footer">
                Don't have an account?
                <a href="${pageContext.request.contextPath}/register.jsp"><strong>Register h
                  ■ ere</strong></a>
            </p>
        </div>
    </main>

    <footer class="footer">
        <p>&copy; 2026 RentRide Inc. Developed with Java Servlets, JSP, JDBC, MySQL & Tomcat
          ■  9.</p>
    </footer>
</body>
</html>
```

File: src/main/webapp/register.jsp | **Lines:** 60 | **Language:** JSP

Description: Customer registration form with real-time validation guidelines.

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,initial-scale=1">
    <title>Create Account | RentRide</title>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body>
    <%@ include file="includes/navbar.jsp" %>

    <main class="main-content auth-wrapper">
        <div class="auth-container">
            <h2>Create Account</h2>
            <p class="subtitle">Join RentRide to reserve cars and motorcycles.</p>

            <% if (request.getAttribute("error") != null) { %>
                <div class="message error">
                    <%= request.getAttribute("error") %>
                </div>
            <% } %>

            <form action="${pageContext.request.contextPath}/register" method="post">
                <div class="form-group">
                    <label for="name">Full Name</label>
                    <input type="text" id="name" name="name"
                           placeholder="Enter your full name"
                           maxlength="100" autocomplete="name" required>
                </div>
                <div class="form-group">
```

```
                    <label for="email">Email Address</label>
                    <input type="email" id="email" name="email"
                           placeholder="name@example.com"
                           maxlength="100" autocomplete="email" required>
                </div>

                <div class="form-group">
                    <label for="password">Password</label>
                    <input type="password" id="password" name="password"
                           placeholder="Minimum 8 characters"
                           minlength="8" autocomplete="new-password" required>
                </div>

                <button type="submit" class="btn btn-primary" style="width:100%; margin-top:
                  ■ 10px;">Create Account</button>
            </form>

            <p class="auth-footer">
                Already have an account?
                <a href="${pageContext.request.contextPath}/login.jsp"><strong>Sign in here<
                  ■ /strong></a>
            </p>
        </div>
    </main>

    <footer class="footer">
        <p>&copy; 2026 RentRide Inc. Developed with Java Servlets, JSP, JDBC, MySQL & Tomcat
          ■  9.</p>
    </footer>
</body>
</html>
```

File: src/main/webapp/booking.jsp | **Lines:** 121 | **Language:** JSP

Description: Interactive reservation form with client-side price calculator and error banner.

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="com.rental.model.Vehicle" %>
<%
    Vehicle vehicle = (Vehicle) request.getAttribute("vehicle");
    if (vehicle == null) {
        response.sendRedirect(request.getContextPath() + "/vehicles");
        return;
    }
%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Book <%= vehicle.getBrand() %> <%= vehicle.getModel() %> | RentRide</title>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body>
    <%@ include file="includes/navbar.jsp" %>

    <main class="main-content">
        <div class="booking-container">
            <div class="booking-card">
                <div class="booking-preview">
                    <img src="${pageContext.request.contextPath}/<%= vehicle.getImageUrl() %
                      ■ >"
                         alt="<%= vehicle.getBrand() %>"
                         class="booking-preview-img"
                         onerror="this.src='${pageContext.request.contextPath}/images/vehicl
                           ■ es/default.jpg'">
                    <div>
                        <span class="vehicle-type-tag"><%= vehicle.getType() %></span>
                        <h2><%= vehicle.getBrand() %> <%= vehicle.getModel() %></h2>
                        <p class="subtitle" style="margin-top:2px;">Reg: <%= vehicle.getVehi
                          ■ cleNumber() %></p>
                        <p style="font-weight:700; color:var(--primary); margin-top:4px;">
                            ■<span id="ratePerDay"><%= String.format("%.2f", vehicle.getPric
                              ■ ePerDay()) %></span> / day
                        </p>
                    </div>
                </div>

                <% if (request.getAttribute("error") != null) { %>
                    <div class="message error">
                        <%= request.getAttribute("error") %>
                    </div>
                <% } %>
                <form action="${pageContext.request.contextPath}/booking" method="post" id="
```

```
                ■ bookingForm">
                  <input type="hidden" name="vehicleId" value="<%= vehicle.getId() %>">

                  <div class="form-row">
                      <div class="form-group">
                          <label for="startDate">Pick-up Date</label>
                          <input type="date" id="startDate" name="startDate" required>
                      </div>

                      <div class="form-group">
                          <label for="endDate">Drop-off Date</label>
                          <input type="date" id="endDate" name="endDate" required>
                      </div>
                  </div>

                  <div class="price-breakdown" id="priceBreakdown" style="display:none;">
                      <div class="breakdown-row">
                          <span>Rental Duration:</span>
                          <strong id="totalDaysText">0 days</strong>
                      </div>
                      <div class="breakdown-row total-row">
                          <span>Estimated Total Amount:</span>
                          <strong id="totalPriceText" style="color:var(--primary);">■0.00<
                            ■ /strong>
                      </div>
                  </div>

                  <div style="display: flex; gap: 12px; margin-top: 24px;">
                      <button type="submit" class="btn btn-primary" style="flex:1;">Procee
                        ■ d to Payment</button>
                      <a href="${pageContext.request.contextPath}/vehicle-details?id=<%= v
                        ■ ehicle.getId() %>" class="btn btn-outline">Cancel</a>
                  </div>
              </form>
          </div>
      </div>
  </main>

  <footer class="footer">
      <p>&copy; 2026 RentRide Inc. Developed with Java Servlets, JSP, JDBC, MySQL & Tomcat
        ■  9.</p>
  </footer>

  <script>
      const ratePerDay = <%= vehicle.getPricePerDay() %>;
      const startInput = document.getElementById('startDate');
      const endInput = document.getElementById('endDate');
      const priceBreakdown = document.getElementById('priceBreakdown');
      const daysText = document.getElementById('totalDaysText');
      const priceText = document.getElementById('totalPriceText');

      const today = new Date().toISOString().split('T')[0];
      startInput.min = today;
      endInput.min = today;

      function updatePrice() {
          if (startInput.value && endInput.value) {
              const start = new Date(startInput.value);
              const end = new Date(endInput.value);
              const diffTime = end - start;
              const diffDays = Math.ceil(diffTime / (1000 * 60 * 60 * 24));

              if (diffDays > 0) {
                  const total = diffDays * ratePerDay;
                  priceBreakdown.style.display = 'block';
                  daysText.textContent = diffDays + (diffDays === 1 ? ' day' : ' days');
                  priceText.textContent = '■' + total.toFixed(2);
              } else {
                  priceBreakdown.style.display = 'none';
              }
          }
      }

      startInput.addEventListener('change', () => {
          endInput.min = startInput.value;
          updatePrice();
      });
      endInput.addEventListener('change', updatePrice);
  </script>
</body>
</html>
```

File: src/main/webapp/payment.jsp	**Lines:** 114 | **Language:** JSP

Description: Development payment sandbox simulating Card, UPI, and Netbanking gateways.

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="com.rental.model.Booking" %>
<%
    Booking booking = (Booking) request.getAttribute("booking");
    if (booking == null) {
        response.sendRedirect(request.getContextPath() + "/my-bookings");
        return;
    }
%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Mock Payment Sandbox | RentRide</title>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body>
    <%@ include file="includes/navbar.jsp" %>

    <main class="main-content">
        <div class="payment-container">
            <div class="mock-banner">
                <span class="mock-badge">TEST SANDBOX</span>
                <p><strong>Development Payment Simulation</strong> — No actual currency or b
                  ■ anking transactions occur.</p>
            </div>

            <% if (request.getParameter("error") != null) { %>
                <div class="message error">
                    <span>■</span> <%= request.getParameter("error") %>
                </div>
            <% } %>

            <div class="checkout-grid">
                <div class="checkout-summary-card">
                    <h2>Reservation Summary</h2>
                    <div class="summary-item">
                        <span style="color:var(--text-muted);">Booking Reference:</span>
                        <strong>#<%= booking.getId() %></strong>
                    </div>
                    <div class="summary-item">
                        <span style="color:var(--text-muted);">Vehicle:</span>
                        <strong><%= booking.getVehicleName() != null ? booking.getVehicleNam
                          ■ e() : ("Vehicle #" + booking.getVehicleId()) %></strong>
                    </div>
                    <div class="summary-item">
                        <span style="color:var(--text-muted);">Rental Window:</span>
                        <span><%= booking.getStartDate() %> &rarr; <%= booking.getEndDate()
                          ■ %></span>
                    </div>
                    <div class="summary-divider"></div>
                    <div class="summary-item" style="font-size: 17px; margin-top: 14px;">
                        <span>Total Payable:</span>
                        <strong class="highlight-price">■<%= String.format("%.2f", booking.g
                          ■ etTotalAmount()) %></strong>
                    </div>
                </div>

                <div class="checkout-actions-card">
                    <h2>Select Mock Payment Method</h2>

                    <form action="${pageContext.request.contextPath}/payment" method="post"
                      ■ id="paymentForm">
                        <input type="hidden" name="bookingId" value="<%= booking.getId() %>"
                          ■ >
                        <input type="hidden" name="paymentStatus" id="paymentStatusInput" va
                          ■ lue="SUCCESS">

                        <div class="mock-methods">
                            <label class="method-option">
                                <input type="radio" name="mockMethod" value="CARD" checked>
                                <div>
                                    <strong style="display:block; font-size:14px;">■ Simulat
                                      ■ ed Credit/Debit Card</strong>
                                    <small style="color:var(--text-muted);">Mock instant car
                                      ■ d gateway</small>
                                </div>
                            </label>

                            <label class="method-option">
                                <input type="radio" name="mockMethod" value="UPI">
                                <div>
```

```
                                <strong style="display:block; font-size:14px;">■ Simulat
                                  ■ ed UPI (Google Pay / PhonePe)</strong>
                                <small style="color:var(--text-muted);">Mock instant UPI
                                  ■  authorization</small>
                            </div>
                        </label>

                        <label class="method-option">
                            <input type="radio" name="mockMethod" value="NETBANKING">
                            <div>
                                <strong style="display:block; font-size:14px;">■ Simulat
                                  ■ ed Net Banking</strong>
                                <small style="color:var(--text-muted);">Mock banking por
                                  ■ tal gateway</small>
                            </div>
                        </label>
                    </div>

                    <div class="payment-action-buttons">
                        <button type="submit" class="btn btn-success" onclick="document.
                          ■ getElementById('paymentStatusInput').value='SUCCESS';">
                            ✓ Authorize Successful Payment (■<%= String.format("%.2f", b
                              ■ ooking.getTotalAmount()) %>)
                        </button>

                        <button type="submit" class="btn btn-danger-outline" onclick="do
                          ■ cument.getElementById('paymentStatusInput').value='FAILED';"
                          ■ >
                            ✗ Simulate Payment Failure
                        </button>
                    </div>

                    <p style="text-align: center; margin-top: 20px;">
                        <a href="${pageContext.request.contextPath}/my-bookings" style="
                          ■ font-size: 13px; color: var(--text-muted);">
                            &larr; Return to My Bookings
                        </a>
                    </p>
                </form>
            </div>
        </div>
    </div>
</main>

<footer class="footer">
    <p>&copy; 2026 RentRide Inc. Developed with Java Servlets, JSP, JDBC, MySQL & Tomcat
      ■  9.</p>
</footer>
</body>
</html>
```

File: src/main/webapp/my-bookings.jsp | **Lines:** 127 | **Language:** JSP

Description: Customer booking dashboard displaying reservation cards and status badges.

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="java.util.List,com.rental.model.Booking" %>
<%
    List<Booking> bookings = (List<Booking>) request.getAttribute("bookings");
    if (bookings == null) {
        response.sendRedirect(request.getContextPath() + "/my-bookings");
        return;
    }
%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My Bookings | RentRide</title>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body>
    <%@ include file="includes/navbar.jsp" %>

    <main class="main-content">
        <div class="content-container">
            <div class="page-header">
                <div>
                    <h1>My Reservations</h1>
                    <p class="subtitle">Manage your current vehicle rentals, payment statuse
                      ■ s, and history.</p>
                </div>
                <a href="${pageContext.request.contextPath}/vehicles" class="btn btn-primary
```

```
      ■ ">+ Rent Another Vehicle</a>
</div>

<% if ("success".equals(request.getParameter("payment"))) { %>
    <div class="message success">
        <span>✓</span> Payment completed successfully! Your vehicle reservation
          ■ is now confirmed.
    </div>
<% } %>

<% if ("cancelled".equals(request.getParameter("msg"))) { %>
    <div class="message success">
        <span>✓</span> Reservation has been successfully cancelled.
    </div>
<% } %>

<% if ("already_confirmed".equals(request.getParameter("info"))) { %>
    <div class="message info">
        <span>■</span> This reservation is already confirmed and active.
    </div>
<% } %>

<% if (request.getParameter("error") != null) { %>
    <div class="message error">
        <span>■</span> <%= request.getParameter("error") %>
    </div>
<% } %>

<div class="table-card">
    <% if (bookings.isEmpty()) { %>
        <div style="text-align: center; padding: 60px 20px;">
            <div style="font-size: 48px; margin-bottom: 12px;">■</div>
            <h2>No Bookings Yet</h2>
            <p class="subtitle" style="margin-bottom: 24px;">You haven't reserve
              ■ d any vehicles with us yet.</p>
            <a href="${pageContext.request.contextPath}/vehicles" class="btn btn
              ■ -primary">Find a Vehicle</a>
        </div>
    <% } else { %>
        <div class="responsive-table-wrapper">
            <table class="data-table">
                <thead>
                    <tr>
                        <th>Ref ID</th>
                        <th>Vehicle</th>
                        <th>Pick-up Date</th>
                        <th>Drop-off Date</th>
                        <th>Total Payable</th>
                        <th>Status</th>
                        <th>Actions</th>
                    </tr>
                </thead>
                <tbody>
                    <% for (Booking b : bookings) { %>
                        <tr>
                            <td><strong>#<%= b.getId() %></strong></td>
                            <td>
                                <div style="font-weight: 700; color: var(--dark)
                                  ■ ;">
                                    <%= b.getVehicleName() != null && !b.getVehi
                                      ■ cleName().isBlank() ? b.getVehicleName()
                                      ■  : ("Vehicle #" + b.getVehicleId()) %>
                                </div>
                                <% if (b.getVehicleNumber() != null) { %>
                                    <div style="font-size: 12px; color: var(--te
                                      ■ xt-muted);"><%= b.getVehicleNumber() %><
                                      ■ /div>
                                <% } %>
                            </td>
                            <td><%= b.getStartDate() %></td>
                            <td><%= b.getEndDate() %></td>
                            <td><strong style="color:var(--dark);">■<%= String.f
                              ■ ormat("%.2f", b.getTotalAmount()) %></strong></t
                              ■ d>
                            <td>
                                <span class="status-badge status-<%= b.getStatus
                                  ■ ().toLowerCase() %>">
                                    <%= b.getStatus() %>
                                </span>
                            </td>
                            <td class="action-cell">
                                <% if ("PENDING".equalsIgnoreCase(b.getStatus())
                                  ■ ) { %>
                                    <a href="${pageContext.request.contextPath}/
                                      ■ payment?bookingId=<%= b.getId() %>" clas
                                      ■ s="btn-sm btn-action-pay">Pay Now</a>
```

```
                                        <form action="${pageContext.request.contextP
                                          ■ ath}/cancel-booking" method="post" style
                                          ■ ="display:inline;" onsubmit="return conf
                                          ■ irm('Cancel this reservation?');">
                                            <input type="hidden" name="bookingId" va
                                              ■ lue="<%= b.getId() %>">
                                            <button type="submit" class="btn-sm btn-
                                              ■ action-cancel">Cancel</button>
                                        </form>
                                    <% } else if ("CONFIRMED".equalsIgnoreCase(b.get
                                      ■ Status())) { %>
                                        <form action="${pageContext.request.contextP
                                          ■ ath}/cancel-booking" method="post" style
                                          ■ ="display:inline;" onsubmit="return conf
                                          ■ irm('Cancel this confirmed reservation?'
                                          ■ );">
                                            <input type="hidden" name="bookingId" va
                                              ■ lue="<%= b.getId() %>">
                                            <button type="submit" class="btn-sm btn-
                                              ■ action-cancel">Cancel</button>
                                        </form>
                                    <% } else { %>
                                        <span style="color:var(--text-light); font-s
                                          ■ ize:13px;">—</span>
                                    <% } %>
                                </td>
                            </tr>
                        <% } %>
                    </tbody>
                </table>
            </div>
        <% } %>
    </div>
</div>
</main>

<footer class="footer">
    <p>&copy; 2026 RentRide Inc. Developed with Java Servlets, JSP, JDBC, MySQL & Tomcat
      ■ 9.</p>
</footer>
</body>
</html>
```

File: src/main/webapp/includes/navbar.jsp | **Lines:** 44 | **Language:** JSP

Description: Reusable navigation header dynamically adapting to guest, user, and admin sessions.

```
<%@ page pageEncoding="UTF-8" %>
<%
    String navUserRole = (String) session.getAttribute("userRole");
    String navUserName = (String) session.getAttribute("userName");
    Integer navUserId = (Integer) session.getAttribute("userId");
    boolean navIsLoggedIn = (navUserId != null);
    boolean navIsAdmin = "ADMIN".equalsIgnoreCase(navUserRole);
%>
<header class="site-header">
    <div class="header-container">
        <a class="nav-brand" href="${pageContext.request.contextPath}/index.jsp">
            <span class="brand-logo-icon">■</span>
            <span class="brand-text">RentRide</span>
        </a>

        <nav class="nav-menu">
            <% if (!navIsLoggedIn) { %>
                <a href="${pageContext.request.contextPath}/index.jsp" class="nav-link">Home
                  ■ </a>
                <a href="${pageContext.request.contextPath}/vehicles" class="nav-link">Brows
                  ■ e Vehicles</a>
                <a href="${pageContext.request.contextPath}/login.jsp" class="nav-link">Logi
                  ■ n</a>
                <a href="${pageContext.request.contextPath}/register.jsp" class="btn btn-pri
                  ■ mary btn-nav">Register</a>
            <% } else if (navIsAdmin) { %>
                <a href="${pageContext.request.contextPath}/index.jsp" class="nav-link">Home
                  ■ </a>
                <a href="${pageContext.request.contextPath}/admin/dashboard" class="nav-link
                  ■ ">Admin Dashboard</a>
                <a href="${pageContext.request.contextPath}/admin/vehicles" class="nav-link"
                  ■ >Manage Vehicles</a>
                <a href="${pageContext.request.contextPath}/admin/bookings" class="nav-link"
                  ■ >Manage Bookings</a>
                <div class="user-pill admin-pill">
                    <span class="role-badge">Admin</span>
                    <span class="user-name"><%= navUserName != null ? navUserName : "Admin"
```

```
                ■ %></span>
            </div>
            <a href="${pageContext.request.contextPath}/logout" class="btn btn-outline b
              ■ tn-nav">Logout</a>
        <% } else { %>
            <a href="${pageContext.request.contextPath}/index.jsp" class="nav-link">Home
              ■ </a>
            <a href="${pageContext.request.contextPath}/vehicles" class="nav-link">Brows
              ■ e Vehicles</a>
            <a href="${pageContext.request.contextPath}/my-bookings" class="nav-link">My
              ■  Bookings</a>
            <div class="user-pill">
                <span class="user-icon">■</span>
                <span class="user-name"><%= navUserName != null ? navUserName : "User" %
                  ■ ></span>
            </div>
            <a href="${pageContext.request.contextPath}/logout" class="btn btn-outline b
              ■ tn-nav">Logout</a>
        <% } %>
    </nav>
</div>
</header>
```

File: src/main/webapp/admin/dashboard.jsp | **Lines:** 125 | **Language:** JSP

Description: Operational analytics dashboard with live KPI cards and quick navigation.

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="java.util.List,com.rental.model.Booking" %>
<%
    Integer totalVehicles = (Integer) request.getAttribute("totalVehicles");
    if (totalVehicles == null) {
        response.sendRedirect(request.getContextPath() + "/admin/dashboard");
        return;
    }
    Integer availableVehicles = (Integer) request.getAttribute("availableVehicles");
    Integer totalBookings = (Integer) request.getAttribute("totalBookings");
    Integer pendingBookings = (Integer) request.getAttribute("pendingBookings");
    Double totalRevenue = (Double) request.getAttribute("totalRevenue");
    List<Booking> recentBookings = (List<Booking>) request.getAttribute("recentBookings");
%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Admin Dashboard | RentRide</title>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body>
    <%@ include file="../includes/navbar.jsp" %>

    <main class="main-content">
        <div class="content-container">
            <div class="page-header">
                <div>
                    <h1>Operational Dashboard</h1>
                    <p class="subtitle">Real-time fleet metrics, revenue performance, and pe
                      ■ nding requests.</p>
                </div>
                <div>
                    <a href="${pageContext.request.contextPath}/admin/vehicles" class="btn b
                      ■ tn-primary">+ Add Vehicle</a>
                </div>
            </div>

            <div class="metrics-grid">
                <div class="metric-card">
                    <span class="metric-title">Total Fleet Units</span>
                    <span class="metric-value"><%= totalVehicles %></span>
                    <span class="metric-sub"><%= availableVehicles %> Available for rent</sp
                      ■ an>
                </div>
                <div class="metric-card">
                    <span class="metric-title">Active / Ready Fleet</span>
                    <span class="metric-value text-success"><%= availableVehicles %></span>
                    <span class="metric-sub">Immediate dispatch ready</span>
                </div>
                <div class="metric-card">
                    <span class="metric-title">Total Bookings</span>
                    <span class="metric-value"><%= totalBookings %></span>
                    <span class="metric-sub"><%= pendingBookings %> Awaiting approval</span>
                </div>
                <div class="metric-card">
```

```
                <span class="metric-title">Pending Approvals</span>
                <span class="metric-value text-warning"><%= pendingBookings %></span>
                <span class="metric-sub">Requires staff review</span>
            </div>
            <div class="metric-card">
                <span class="metric-title">Gross Revenue</span>
                <span class="metric-value text-primary">■<%= String.format("%.2f", total
                  ■ Revenue != null ? totalRevenue : 0.0) %></span>
                <span class="metric-sub">Settled transactions</span>
            </div>
        </div>

        <div style="margin-top: 36px;">
            <div style="display:flex; justify-content:space-between; align-items:center;
              ■  margin-bottom: 16px;">
                <h2 style="font-size: 20px; font-weight: 700; color: var(--dark);">Recen
                  ■ t Reservations</h2>
                <a href="${pageContext.request.contextPath}/admin/bookings" style="font-
                  ■ size: 13px; font-weight: 600;">View All Bookings &rarr;</a>
            </div>

            <div class="table-card">
                <% if (recentBookings == null || recentBookings.isEmpty()) { %>
                    <div style="text-align: center; padding: 40px; color: var(--text-mut
                      ■ ed);">
                        No reservations recorded yet.
                    </div>
                <% } else { %>
                    <div class="responsive-table-wrapper">
                        <table class="data-table">
                            <thead>
                                <tr>
                                    <th>Ref ID</th>
                                    <th>Customer</th>
                                    <th>Vehicle</th>
                                    <th>Dates</th>
                                    <th>Total (■)</th>
                                    <th>Status</th>
                                    <th>Action</th>
                                </tr>
                            </thead>
                            <tbody>
                                <% for (Booking b : recentBookings) { %>
                                    <tr>
                                        <td><strong>#<%= b.getId() %></strong></td>
                                        <td>
                                            <strong><%= b.getUserName() != null ? b.getU
                                              ■ serName() : "User #" + b.getUserId() %><
                                              ■ /strong><br>
                                            <small style="color:var(--text-muted);"><%=
                                              ■ b.getUserEmail() != null ? b.getUserEmai
                                              ■ l() : "" %></small>
                                        </td>
                                        <td><%= b.getVehicleName() != null ? b.getVehicl
                                          ■ eName() : "Vehicle #" + b.getVehicleId() %><
                                          ■ /td>
                                        <td><%= b.getStartDate() %> &rarr; <%= b.getEndD
                                          ■ ate() %></td>
                                        <td><strong>■<%= String.format("%.2f", b.getTota
                                          ■ lAmount()) %></strong></td>
                                        <td>
                                            <span class="status-badge status-<%= b.getSt
                                              ■ atus().toLowerCase() %>">
                                               <%= b.getStatus() %>
                                            </span>
                                        </td>
                                        <td>
                                            <a href="${pageContext.request.contextPath}/
                                              ■ admin/bookings" class="btn-sm">Manage</a
                                              ■ >
                                        </td>
                                    </tr>
                                <% } %>
                            </tbody>
                        </table>
                    </div>
                <% } %>
            </div>
        </div>
    </div>
</main>

<footer class="footer">
    <p>&copy; 2026 RentRide Inc. Developed with Java Servlets, JSP, JDBC, MySQL & Tomcat
      ■  9.</p>
</footer>
```

```
</body>
</html>
```

File: src/main/webapp/admin/vehicles.jsp | **Lines:** 210 | **Language:** JSP

Description: Fleet inventory management table with status toggles and Add Vehicle modal.

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="java.util.List,com.rental.model.Vehicle" %>
<%
    List<Vehicle> vehicles = (List<Vehicle>) request.getAttribute("vehicles");
    if (vehicles == null) {
        response.sendRedirect(request.getContextPath() + "/admin/vehicles");
        return;
    }
%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Manage Fleet | RentRide Admin</title>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body>
    <%@ include file="../includes/navbar.jsp" %>

    <main class="main-content">
        <div class="content-container">
            <div class="page-header">
                <div>
                    <h1>Fleet Inventory Management</h1>
                    <p class="subtitle">Add new rental units, adjust daily rates, and contro
                      ■ l availability.</p>
                </div>
                <button type="button" class="btn btn-primary" onclick="toggleAddForm()">+ Ad
                  ■ d New Vehicle</button>
            </div>

            <!-- Add Vehicle Form Card -->
            <div class="table-card" id="addVehicleForm" style="display:none; padding: 28px;
              ■ margin-bottom: 28px;">
                <h3 style="margin-bottom: 18px; color: var(--dark);">Add New Vehicle to Flee
                  ■ t</h3>
                <form action="${pageContext.request.contextPath}/admin/vehicles" method="pos
                  ■ t">
                    <input type="hidden" name="action" value="add">

                    <div class="form-row">
                        <div class="form-group">
                            <label for="vehicleNumber">Registration Number</label>
                            <input type="text" id="vehicleNumber" name="vehicleNumber" place
                              ■ holder="e.g. AP39XY9999" required>
                        </div>
                        <div class="form-group">
                            <label for="brand">Brand / Make</label>
                            <input type="text" id="brand" name="brand" placeholder="e.g. Mar
                              ■ uti / Hyundai" required>
                        </div>
                        <div class="form-group">
                            <label for="model">Model Name</label>
                            <input type="text" id="model" name="model" placeholder="e.g. Swi
                              ■ ft / Creta" required>
                        </div>
                    </div>

                    <div class="form-row">
                        <div class="form-group">
                            <label for="type">Vehicle Class</label>
                            <select id="type" name="type" required class="form-select">
                                <option value="Hatchback">Hatchback</option>
                                <option value="Sedan" selected>Sedan</option>
                                <option value="SUV">SUV</option>
                                <option value="Luxury">Luxury</option>
                                <option value="Bike">Motorcycle / Scooter</option>
                            </select>
                        </div>
                        <div class="form-group">
                            <label for="pricePerDay">Daily Rate (■)</label>
                            <input type="number" id="pricePerDay" name="pricePerDay" step="0
                              ■ .01" min="1" placeholder="e.g. 1800.00" required>
                        </div>
                    </div>
                    <div style="display:flex; gap:12px; margin-top:16px;">
```

```
            <button type="submit" class="btn btn-success">Save to Fleet</button>
            <button type="button" class="btn btn-secondary" onclick="toggleAddFo
              ■ rm()">Cancel</button>
        </div>
    </form>
</div>

<!-- Vehicles Table -->
<div class="table-card">
    <div class="responsive-table-wrapper">
        <table class="data-table">
            <thead>
                <tr>
                    <th>Photo</th>
                    <th>Vehicle</th>
                    <th>Registration</th>
                    <th>Category</th>
                    <th>Rate / Day</th>
                    <th>Availability Status</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                <% for (Vehicle v : vehicles) { %>
                    <tr>
                        <td>
                            <img src="${pageContext.request.contextPath}/<%= v.g
                              ■ etImageUrl() %>"
                                 alt="<%= v.getBrand() %>"
                                 style="width: 72px; height: 46px; object-fit: c
                                   ■ over; border-radius: 6px; border: 1px solid
                                   ■  var(--border);"
                                 onerror="this.src='${pageContext.request.contex
                                   ■ tPath}/images/vehicles/default.jpg'">
                        </td>
                        <td>
                            <strong><%= v.getBrand() %> <%= v.getModel() %></str
                              ■ ong>
                        </td>
                        <td><code><%= v.getVehicleNumber() %></code></td>
                        <td><%= v.getType() %></td>
                        <td><strong>■<%= String.format("%.2f", v.getPricePerDay(
                          ■ )) %></strong></td>
                        <td>
                            <form action="${pageContext.request.contextPath}/adm
                              ■ in/vehicles" method="post" style="display:inline
                              ■ ;">
                                <input type="hidden" name="action" value="toggle
                                  ■ Status">
                                <input type="hidden" name="id" value="<%= v.getI
                                  ■ d() %>">
                                <button type="submit" class="status-toggle-btn <
                                  ■ %= "AVAILABLE".equalsIgnoreCase(v.getStatus(
                                  ■ )) ? "status-available" : "status-unavailabl
                                  ■ e" %>" title="Click to toggle status">
                                    <%= v.getStatus() %>
                                </button>
                            </form>
                        </td>
                        <td class="action-cell">
                            <button type="button" class="btn-sm btn-action-edit"
                                    onclick="openEditModal(<%= v.getId() %>, '<%
                                      ■ = v.getVehicleNumber() %>', '<%= v.getBr
                                      ■ and() %>', '<%= v.getModel() %>', '<%= v
                                      ■ .getType() %>', <%= v.getPricePerDay() %
                                      ■ >, '<%= v.getStatus() %>')">
                                Edit
                            </button>

                            <form action="${pageContext.request.contextPath}/adm
                              ■ in/vehicles" method="post" style="display:inline
                              ■ ;" onsubmit="return confirm('Delete vehicle <%=
                              ■ v.getBrand() %> <%= v.getModel() %>?');">
                                <input type="hidden" name="action" value="delete
                                  ■ ">
                                <input type="hidden" name="id" value="<%= v.getI
                                  ■ d() %>">
                                <button type="submit" class="btn-sm btn-action-d
                                  ■ elete">Delete</button>
                            </form>
                        </td>
                    </tr>
                <% } %>
            </tbody>
        </table>
    </div>
```

```
            </div>
        </div>
    </main>

    <!-- Edit Vehicle Modal -->
    <div id="editModal" class="modal-overlay" style="display:none;">
        <div class="modal-card">
            <h3 style="margin-bottom: 18px; color: var(--dark);">Edit Vehicle Details</h3>
            <form action="${pageContext.request.contextPath}/admin/vehicles" method="post">
                <input type="hidden" name="action" value="update">
                <input type="hidden" name="id" id="editId">

                <div class="form-group">
                    <label for="editNumber">Registration Number</label>
                    <input type="text" id="editNumber" name="vehicleNumber" required>
                </div>
                <div class="form-row">
                    <div class="form-group">
                        <label for="editBrand">Brand</label>
                        <input type="text" id="editBrand" name="brand" required>
                    </div>
                    <div class="form-group">
                        <label for="editModel">Model</label>
                        <input type="text" id="editModel" name="model" required>
                    </div>
                </div>
                <div class="form-row">
                    <div class="form-group">
                        <label for="editType">Category</label>
                        <input type="text" id="editType" name="type" required>
                    </div>
                    <div class="form-group">
                        <label for="editPrice">Daily Rate (■)</label>
                        <input type="number" id="editPrice" name="pricePerDay" step="0.01" r
                          ■ equired>
                    </div>
                </div>
                <div class="form-group">
                    <label for="editStatus">Status</label>
                    <select id="editStatus" name="status" class="form-select">
                        <option value="AVAILABLE">AVAILABLE</option>
                        <option value="UNAVAILABLE">UNAVAILABLE</option>
                    </select>
                </div>

                <div class="modal-actions">
                    <button type="submit" class="btn btn-primary" style="flex:1;">Update Veh
                      ■ icle</button>
                    <button type="button" class="btn btn-secondary" onclick="closeEditModal(
                      ■ )">Cancel</button>
                </div>
            </form>
        </div>
    </div>

    <footer class="footer">
        <p>&copy; 2026 RentRide Inc. Developed with Java Servlets, JSP, JDBC, MySQL & Tomcat
          ■  9.</p>
    </footer>

    <script>
        function toggleAddForm() {
            const form = document.getElementById('addVehicleForm');
            form.style.display = (form.style.display === 'none') ? 'block' : 'none';
        }

        function openEditModal(id, number, brand, model, type, price, status) {
            document.getElementById('editId').value = id;
            document.getElementById('editNumber').value = number;
            document.getElementById('editBrand').value = brand;
            document.getElementById('editModel').value = model;
            document.getElementById('editType').value = type;
            document.getElementById('editPrice').value = price;
            document.getElementById('editStatus').value = status;
            document.getElementById('editModal').style.display = 'flex';
        }

        function closeEditModal() {
            document.getElementById('editModal').style.display = 'none';
        }
    </script>
</body>
</html>
```

File: src/main/webapp/admin/bookings.jsp	**Lines:** 111 | **Language:** JSP
Description: Customer reservations management table with administrative action triggers.	

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="java.util.List,com.rental.model.Booking" %>
<%
    List<Booking> bookings = (List<Booking>) request.getAttribute("bookings");
    if (bookings == null) {
        response.sendRedirect(request.getContextPath() + "/admin/bookings");
        return;
    }
%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Manage Bookings | RentRide Admin</title>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body>
    <%@ include file="../includes/navbar.jsp" %>

    <main class="main-content">
        <div class="content-container">
            <div class="page-header">
                <div>
                    <h1>Customer Reservations Control</h1>
                    <p class="subtitle">Review booking requests, manual status overrides, an
                      ■ d booking history.</p>
                </div>
            </div>

            <div class="table-card">
                <% if (bookings.isEmpty()) { %>
                    <div style="text-align: center; padding: 40px; color: var(--text-muted);
                      ■ ">
                        No customer reservations recorded yet.
                    </div>
                <% } else { %>
                    <div class="responsive-table-wrapper">
                        <table class="data-table">
                            <thead>
                                <tr>
                                    <th>Ref ID</th>
                                    <th>Customer Details</th>
                                    <th>Reserved Vehicle</th>
                                    <th>Rental Window</th>
                                    <th>Total (■)</th>
                                    <th>Status</th>
                                    <th>Staff Actions</th>
                                </tr>
                            </thead>
                            <tbody>
                                <% for (Booking b : bookings) { %>
                                    <tr>
                                        <td><strong>#<%= b.getId() %></strong></td>
                                        <td>
                                            <strong style="color:var(--dark);"><%= b.getUser
                                              ■ Name() != null ? b.getUserName() : "User #"
                                              ■ + b.getUserId() %></strong><br>
                                            <small style="color:var(--text-muted);"><%= b.ge
                                              ■ tUserEmail() != null ? b.getUserEmail() : ""
                                              ■  %></small>
                                        </td>
                                        <td>
                                            <strong><%= b.getVehicleName() != null ? b.getVe
                                              ■ hicleName() : "Vehicle #" + b.getVehicleId()
                                              ■  %></strong>
                                            <% if (b.getVehicleNumber() != null) { %>
                                                <br><small style="color:var(--text-muted); f
                                                  ■ ont-size:12px;"><%= b.getVehicleNumber()
                                                  ■  %></small>
                                            <% } %>
                                        </td>
                                        <td>
                                            <span><%= b.getStartDate() %></span><br>
                                            <small style="color:var(--text-muted);">&darr; t
                                              ■ o</small><br>
                                            <span><%= b.getEndDate() %></span>
                                        </td>
                                        <td><strong style="color:var(--dark);">■<%= String.f
                                          ■ ormat("%.2f", b.getTotalAmount()) %></strong></t
                                          ■ d>
```

```
                                        <td>
                                            <span class="status-badge status-<%= b.getStatus
                                              ■ ().toLowerCase() %>">
                                                <%= b.getStatus() %>
                                            </span>
                                        </td>
                                        <td class="action-cell">
                                            <% if ("PENDING".equalsIgnoreCase(b.getStatus())
                                              ■ ) { %>
                                                <form action="${pageContext.request.contextP
                                                  ■ ath}/admin/bookings" method="post" style
                                                  ■ ="display:inline;">
                                                    <input type="hidden" name="bookingId" va
                                                      ■ lue="<%= b.getId() %>">
                                                    <input type="hidden" name="status" value
                                                      ■ ="CONFIRMED">
                                                    <button type="submit" class="btn-sm btn-
                                                      ■ action-pay">Approve</button>
                                                </form>

                                                <form action="${pageContext.request.contextP
                                                  ■ ath}/admin/bookings" method="post" style
                                                  ■ ="display:inline;">
                                                    <input type="hidden" name="bookingId" va
                                                      ■ lue="<%= b.getId() %>">
                                                    <input type="hidden" name="status" value
                                                      ■ ="REJECTED">
                                                    <button type="submit" class="btn-sm btn-
                                                      ■ action-delete">Reject</button>
                                                </form>
                                            <% } else if ("CONFIRMED".equalsIgnoreCase(b.get
                                              ■ Status())) { %>
                                                <form action="${pageContext.request.contextP
                                                  ■ ath}/admin/bookings" method="post" style
                                                  ■ ="display:inline;">
                                                    <input type="hidden" name="bookingId" va
                                                      ■ lue="<%= b.getId() %>">
                                                    <input type="hidden" name="status" value
                                                      ■ ="CANCELLED">
                                                    <button type="submit" class="btn-sm btn-
                                                      ■ action-cancel">Cancel</button>
                                                </form>
                                            <% } else { %>
                                                <span style="color:var(--text-light); font-s
                                                  ■ ize:13px;">—</span>
                                            <% } %>
                                        </td>
                                    </tr>
                                <% } %>
                            </tbody>
                        </table>
                    </div>
                <% } %>
            </div>
        </div>
    </main>

    <footer class="footer">
        <p>&copy; 2026 RentRide Inc. Developed with Java Servlets, JSP, JDBC, MySQL & Tomcat
          ■ 9.</p>
    </footer>
</body>
</html>
```

D.7 Custom CSS Design System

Production-grade responsive stylesheet featuring glassmorphism and modern tokens

File: src/main/webapp/css/style.css	**Lines:** 1068 | **Language:** CSS
Description: Complete custom stylesheet with CSS variables, responsive grid, and UI components.	

```
/* ======================================================
   RentRide - Comprehensive Modern CSS Design System
   Unified Styling for Customer & Admin Portals
   ====================================================== */

:root {
    --primary: #2563eb;
    --primary-hover: #1d4ed8;
    --primary-light: #eff6ff;
    --primary-glow: rgba(37, 99, 235, 0.18);
```

```
    --secondary: #64748b;
    --secondary-hover: #475569;
    --secondary-light: #f1f5f9;

    --success: #16a34a;
    --success-hover: #15803d;
    --success-light: #f0fdf4;
    --success-border: #bbf7d0;

    --warning: #d97706;
    --warning-light: #fffbeb;
    --warning-border: #fde68a;

    --danger: #dc2626;
    --danger-hover: #b91c1c;
    --danger-light: #fef2f2;
    --danger-border: #fecaca;

    --dark: #0f172a;
    --dark-muted: #1e293b;
    --light-bg: #f8fafc;
    --card-bg: #ffffff;
    --border: #e2e8f0;
    --border-dark: #cbd5e1;

    --text-main: #0f172a;
    --text-muted: #64748b;
    --text-light: #94a3b8;

    --radius-sm: 8px;
    --radius-md: 12px;
    --radius-lg: 18px;
    --radius-full: 9999px;

    --shadow-sm: 0 1px 3px rgba(15, 23, 42, 0.05);
    --shadow-md: 0 4px 16px -2px rgba(15, 23, 42, 0.08);
    --shadow-lg: 0 12px 32px -4px rgba(15, 23, 42, 0.12);
}

/* ================= Base & Reset ================= */
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Ar
      ■ ial, sans-serif;
    background-color: var(--light-bg);
    color: var(--text-main);
    line-height: 1.6;
    min-height: 100vh;
    display: flex;
    flex-direction: column;
}

img {
    max-width: 100%;
    height: auto;
    display: block;
}

a {
    color: var(--primary);
    text-decoration: none;
    transition: color 0.15s ease;
}

a:hover {
    color: var(--primary-hover);
}

/* ================= Global Header / Top Navigation Bar ================= */
.site-header {
    background: #ffffff;
    border-bottom: 1px solid var(--border);
    position: sticky;
    top: 0;
    z-index: 1000;
    box-shadow: 0 2px 10px rgba(15, 23, 42, 0.03);
    width: 100%;
}

.header-container {
    max-width: 1280px;
```

```
    margin: 0 auto;
    padding: 14px 24px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 20px;
}

.nav-brand {
    display: inline-flex;
    align-items: center;
    gap: 10px;
    text-decoration: none;
    color: var(--dark);
    font-size: 22px;
    font-weight: 800;
    letter-spacing: -0.5px;
}

.brand-logo-icon {
    font-size: 26px;
    line-height: 1;
}

.brand-text {
    background: linear-gradient(135deg, #2563eb, #1d4ed8);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}

.nav-menu {
    display: flex;
    align-items: center;
    gap: 20px;
    flex-wrap: wrap;
}

.nav-link {
    font-size: 14px;
    font-weight: 600;
    color: var(--text-muted);
    padding: 8px 12px;
    border-radius: var(--radius-sm);
    transition: all 0.2s ease;
    text-decoration: none;
}

.nav-link:hover,
.nav-link.active {
    color: var(--primary);
    background: var(--primary-light);
}

.user-pill {
    display: inline-flex;
    align-items: center;
    gap: 8px;
    background: var(--secondary-light);
    border: 1px solid var(--border);
    padding: 6px 14px;
    border-radius: var(--radius-full);
    font-size: 13px;
    font-weight: 600;
    color: var(--text-main);
}

.user-pill.admin-pill {
    background: #f8fafc;
    border-color: #cbd5e1;
}

.role-badge {
    background: var(--dark);
    color: #ffffff;
    font-size: 10px;
    font-weight: 800;
    text-transform: uppercase;
    padding: 2px 7px;
    border-radius: 4px;
    letter-spacing: 0.5px;
}

/* ================= Buttons ================= */
.btn {
    display: inline-flex;
    align-items: center;
```

```
    justify-content: center;
    gap: 8px;
    padding: 10px 20px;
    font-size: 14px;
    font-weight: 600;
    border-radius: var(--radius-sm);
    border: 1px solid transparent;
    cursor: pointer;
    text-decoration: none;
    transition: all 0.2s cubic-bezier(0.4, 0, 0.2, 1);
    line-height: 1.4;
}

.btn:focus-visible {
    outline: none;
    box-shadow: 0 0 0 3px var(--primary-glow);
}

.btn:disabled {
    opacity: 0.6;
    cursor: not-allowed;
}

.btn-primary {
    background-color: var(--primary);
    color: #ffffff;
    box-shadow: 0 2px 6px rgba(37, 99, 235, 0.25);
}

.btn-primary:hover {
    background-color: var(--primary-hover);
    color: #ffffff;
    transform: translateY(-1px);
    box-shadow: 0 4px 12px rgba(37, 99, 235, 0.35);
}

.btn-secondary {
    background-color: var(--secondary-light);
    color: var(--text-main);
    border-color: var(--border);
}

.btn-secondary:hover {
    background-color: #e2e8f0;
    color: var(--dark);
}

.btn-outline {
    background-color: transparent;
    border-color: var(--border);
    color: var(--text-muted);
}

.btn-outline:hover {
    background-color: var(--light-bg);
    border-color: var(--border-dark);
    color: var(--text-main);
}

.btn-success {
    background-color: var(--success);
    color: #ffffff;
    box-shadow: 0 2px 6px rgba(22, 163, 74, 0.25);
}

.btn-success:hover {
    background-color: var(--success-hover);
    color: #ffffff;
    transform: translateY(-1px);
}

.btn-danger {
    background-color: var(--danger);
    color: #ffffff;
}

.btn-danger:hover {
    background-color: var(--danger-hover);
    color: #ffffff;
}

.btn-danger-outline {
    background-color: transparent;
    border-color: var(--danger-border);
    color: var(--danger);
}
```

```
.btn-danger-outline:hover {
    background-color: var(--danger-light);
    border-color: var(--danger);
}

.btn-nav {
    padding: 7px 16px;
    font-size: 13px;
}

.btn-sm {
    padding: 6px 12px;
    font-size: 12px;
    font-weight: 600;
    border-radius: var(--radius-sm);
    border: 1px solid var(--border);
    background: #ffffff;
    color: var(--text-main);
    cursor: pointer;
    text-decoration: none;
    transition: all 0.15s ease;
}

.btn-sm:hover {
    background: var(--light-bg);
    border-color: var(--border-dark);
}

.btn-action-pay {
    background-color: var(--primary);
    color: #ffffff;
    border-color: var(--primary);
}

.btn-action-pay:hover {
    background-color: var(--primary-hover);
    color: #ffffff;
}

.btn-action-cancel,
.btn-action-delete {
    background-color: var(--danger-light);
    color: var(--danger);
    border-color: var(--danger-border);
}

.btn-action-cancel:hover,
.btn-action-delete:hover {
    background-color: #fee2e2;
}

.btn-action-edit {
    background-color: var(--primary-light);
    color: var(--primary);
    border-color: #bfdbfe;
}

/* ================= Containers & Page Layouts ================= */
.main-content {
    flex: 1;
    width: 100%;
}

.content-container {
    max-width: 1240px;
    width: 92%;
    margin: 40px auto;
}

.page-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 28px;
    flex-wrap: wrap;
    gap: 16px;
}

.page-header h1 {
    font-size: 28px;
    font-weight: 800;
    color: var(--dark);
    letter-spacing: -0.5px;
}

.subtitle {
```

```
    color: var(--text-muted);
    font-size: 14px;
    margin-top: 4px;
}

/* ================= Status Badges ================= */
.status-badge {
    display: inline-block;
    padding: 4px 10px;
    font-size: 11px;
    font-weight: 700;
    border-radius: var(--radius-full);
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

.status-pending {
    background-color: var(--warning-light);
    color: var(--warning);
    border: 1px solid var(--warning-border);
}

.status-confirmed,
.status-available {
    background-color: var(--success-light);
    color: var(--success);
    border: 1px solid var(--success-border);
}

.status-cancelled,
.status-unavailable,
.status-rejected {
    background-color: var(--danger-light);
    color: var(--danger);
    border: 1px solid var(--danger-border);
}

/* ================= Alert Messages ================= */
.message {
    padding: 12px 18px;
    border-radius: var(--radius-sm);
    font-size: 14px;
    font-weight: 500;
    margin-bottom: 22px;
    line-height: 1.5;
    display: flex;
    align-items: center;
    gap: 10px;
}

.message.error {
    background: var(--danger-light);
    color: var(--danger);
    border: 1px solid var(--danger-border);
}

.message.success {
    background: var(--success-light);
    color: var(--success);
    border: 1px solid var(--success-border);
}

.message.info {
    background: var(--primary-light);
    color: var(--primary);
    border: 1px solid #bfdbfe;
}

/* ================= Forms & Auth Styling ================= */
.auth-wrapper {
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 50px 20px;
    flex: 1;
}

.auth-container {
    max-width: 440px;
    width: 100%;
    padding: 38px;
    background: var(--card-bg);
    border-radius: var(--radius-lg);
    box-shadow: var(--shadow-md);
    border: 1px solid var(--border);
}
```

```
.auth-container h2 {
    font-size: 24px;
    font-weight: 800;
    color: var(--dark);
    text-align: center;
    margin-bottom: 6px;
}

.auth-container .subtitle {
    text-align: center;
    margin-bottom: 26px;
}

.form-group {
    margin-bottom: 18px;
}

.form-group label {
    display: block;
    font-size: 13px;
    font-weight: 600;
    margin-bottom: 7px;
    color: #334155;
}

.form-group input,
.form-select {
    width: 100%;
    padding: 12px 14px;
    border: 1px solid var(--border);
    border-radius: var(--radius-sm);
    font-size: 14px;
    outline: none;
    background: #ffffff;
    color: var(--text-main);
    transition: all 0.2s ease;
}

.form-group input:focus,
.form-select:focus {
    border-color: var(--primary);
    box-shadow: 0 0 0 3px var(--primary-glow);
}

.form-row {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 16px;
}

.auth-footer {
    margin-top: 24px;
    text-align: center;
    font-size: 14px;
    color: var(--text-muted);
}

/* ================= Tables ================= */
.table-card {
    background: var(--card-bg);
    border-radius: var(--radius-md);
    border: 1px solid var(--border);
    box-shadow: var(--shadow-sm);
    overflow: hidden;
}

.responsive-table-wrapper {
    width: 100%;
    overflow-x: auto;
}

.data-table {
    width: 100%;
    border-collapse: collapse;
    text-align: left;
}

.data-table th {
    background: #f8fafc;
    padding: 14px 18px;
    font-size: 12px;
    font-weight: 700;
    text-transform: uppercase;
    color: var(--text-muted);
    border-bottom: 1px solid var(--border);
    letter-spacing: 0.5px;
```

```
}

.data-table td {
    padding: 16px 18px;
    font-size: 14px;
    border-bottom: 1px solid #f1f5f9;
    vertical-align: middle;
}

.data-table tr:hover {
    background-color: #fafbfd;
}

.action-cell {
    white-space: nowrap;
    display: flex;
    gap: 8px;
    align-items: center;
}

/* ================= Vehicle Cards & Image Assets ================= */
.vehicles-main {
    max-width: 1240px;
    width: 92%;
    margin: 40px auto;
}

.vehicles-main h1 {
    font-size: 32px;
    font-weight: 800;
    text-align: center;
    color: var(--dark);
    letter-spacing: -0.5px;
}

.vehicles-main .subtitle {
    text-align: center;
    margin-bottom: 40px;
}

.vehicle-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
    gap: 28px;
}

.vehicle-card {
    background: var(--card-bg);
    border: 1px solid var(--border);
    border-radius: var(--radius-md);
    overflow: hidden;
    box-shadow: var(--shadow-sm);
    transition: transform 0.25s ease, box-shadow 0.25s ease;
    display: flex;
    flex-direction: column;
}

.vehicle-card:hover {
    transform: translateY(-5px);
    box-shadow: var(--shadow-lg);
}

.vehicle-image-wrapper {
    position: relative;
    width: 100%;
    height: 190px;
    background: #f1f5f9;
    overflow: hidden;
}

.vehicle-image-wrapper img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    object-position: center;
    transition: transform 0.35s ease;
}

.vehicle-card:hover .vehicle-image-wrapper img {
    transform: scale(1.05);
}

.vehicle-info {
    padding: 22px;
    display: flex;
    flex-direction: column;
```

```
    flex-grow: 1;
}

.vehicle-type-tag {
    display: inline-block;
    padding: 3px 10px;
    background: var(--primary-light);
    color: var(--primary);
    font-size: 11px;
    font-weight: 700;
    border-radius: var(--radius-full);
    margin-bottom: 10px;
    align-self: flex-start;
}

.vehicle-info h2 {
    font-size: 20px;
    font-weight: 700;
    color: var(--dark);
    margin-bottom: 4px;
}

.vehicle-number {
    font-size: 13px;
    color: var(--text-muted);
    margin-bottom: 16px;
}

.vehicle-price {
    font-size: 24px;
    font-weight: 800;
    color: var(--dark);
    margin-top: auto;
    margin-bottom: 18px;
}

.vehicle-price span {
    font-size: 13px;
    font-weight: 500;
    color: var(--text-muted);
}

.vehicle-info .btn {
    width: 100%;
}

/* ================= Vehicle Details Page ================= */
.details-container {
    max-width: 980px;
    width: 92%;
    margin: 50px auto;
}

.details-card {
    display: grid;
    grid-template-columns: 1.1fr 1fr;
    background: var(--card-bg);
    border-radius: var(--radius-lg);
    border: 1px solid var(--border);
    overflow: hidden;
    box-shadow: var(--shadow-md);
}

.details-image-wrapper {
    width: 100%;
    min-height: 380px;
    background: #f1f5f9;
    overflow: hidden;
}

.details-image-wrapper img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    object-position: center;
}

.details-info {
    padding: 40px;
    display: flex;
    flex-direction: column;
    justify-content: center;
}

.details-info h1 {
    font-size: 30px;
```

```
    font-weight: 800;
    color: var(--dark);
    margin: 8px 0 12px;
}

.details-info .vehicle-meta {
    font-size: 14px;
    color: var(--text-muted);
    margin-bottom: 10px;
}

.details-info .vehicle-price {
    margin: 20px 0;
}

/* ================= Booking Page ================= */
.booking-container {
    max-width: 640px;
    width: 92%;
    margin: 40px auto;
}

.booking-card {
    background: var(--card-bg);
    padding: 38px;
    border-radius: var(--radius-lg);
    border: 1px solid var(--border);
    box-shadow: var(--shadow-md);
}

.booking-preview {
    display: flex;
    gap: 18px;
    align-items: center;
    background: var(--light-bg);
    padding: 16px;
    border-radius: var(--radius-md);
    border: 1px solid var(--border);
    margin-bottom: 24px;
}

.booking-preview-img {
    width: 110px;
    height: 75px;
    border-radius: var(--radius-sm);
    object-fit: cover;
}

.price-breakdown {
    background: var(--light-bg);
    padding: 18px;
    border-radius: var(--radius-sm);
    border: 1px solid var(--border);
    margin: 22px 0;
}

.breakdown-row {
    display: flex;
    justify-content: space-between;
    margin-bottom: 8px;
    font-size: 14px;
}

.total-row {
    margin-top: 10px;
    padding-top: 10px;
    border-top: 1px solid var(--border);
    font-size: 18px;
    font-weight: 800;
    color: var(--dark);
}

/* ================= Mock Payment Checkout ================= */
.payment-container {
    max-width: 860px;
    width: 92%;
    margin: 40px auto;
}

.mock-banner {
    background: #fefce8;
    border: 1px solid var(--warning-border);
    padding: 14px 20px;
    border-radius: var(--radius-sm);
    display: flex;
    align-items: center;
```

```
    gap: 12px;
    margin-bottom: 24px;
}

.mock-badge {
    background: var(--warning);
    color: #ffffff;
    font-size: 11px;
    font-weight: 800;
    padding: 3px 8px;
    border-radius: 4px;
    letter-spacing: 0.5px;
}

.checkout-grid {
    display: grid;
    grid-template-columns: 1fr 1.25fr;
    gap: 28px;
}

.checkout-summary-card,
.checkout-actions-card {
    background: var(--card-bg);
    padding: 30px;
    border-radius: var(--radius-md);
    border: 1px solid var(--border);
    box-shadow: var(--shadow-sm);
}

.checkout-summary-card h2,
.checkout-actions-card h2 {
    font-size: 18px;
    font-weight: 700;
    color: var(--dark);
    margin-bottom: 18px;
    padding-bottom: 10px;
    border-bottom: 1px solid var(--border);
}

.summary-item {
    display: flex;
    justify-content: space-between;
    margin-bottom: 12px;
    font-size: 14px;
}

.summary-divider {
    height: 1px;
    background: var(--border);
    margin: 16px 0;
}

.highlight-price {
    color: var(--primary);
    font-size: 22px;
    font-weight: 800;
}

.mock-methods {
    display: flex;
    flex-direction: column;
    gap: 12px;
    margin-bottom: 24px;
}

.method-option {
    display: flex;
    align-items: center;
    padding: 12px 16px;
    border: 1px solid var(--border);
    border-radius: var(--radius-sm);
    cursor: pointer;
    transition: all 0.2s ease;
}

.method-option:hover {
    border-color: var(--primary);
    background: var(--light-bg);
}

.method-option input {
    margin-right: 12px;
}

.payment-action-buttons {
    display: flex;
```

```
    flex-direction: column;
    gap: 12px;
}

.payment-action-buttons button {
    width: 100%;
    padding: 12px;
}

/* ================= Admin Portal & Dashboard ================= */
.metrics-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 20px;
    margin-bottom: 32px;
}

.metric-card {
    background: var(--card-bg);
    padding: 24px;
    border-radius: var(--radius-md);
    border: 1px solid var(--border);
    box-shadow: var(--shadow-sm);
    display: flex;
    flex-direction: column;
}

.metric-title {
    font-size: 13px;
    font-weight: 600;
    color: var(--text-muted);
    margin-bottom: 8px;
}

.metric-value {
    font-size: 28px;
    font-weight: 800;
    color: var(--dark);
    line-height: 1.1;
    margin-bottom: 6px;
}

.metric-sub {
    font-size: 12px;
    color: var(--text-muted);
}

.text-success { color: var(--success); }
.text-warning { color: var(--warning); }
.text-primary { color: var(--primary); }

.status-toggle-btn {
    border: none;
    cursor: pointer;
    font-size: 11px;
    font-weight: 700;
    padding: 4px 10px;
    border-radius: var(--radius-full);
}

/* ================= Landing Page Hero ================= */
.landing-main {
    max-width: 1140px;
    width: 90%;
    margin: 40px auto 60px;
}

.hero-section {
    text-align: center;
    padding: 50px 20px 40px;
}

.badge {
    display: inline-block;
    padding: 6px 14px;
    background: var(--primary-light);
    color: var(--primary);
    font-size: 13px;
    font-weight: 700;
    border-radius: var(--radius-full);
    margin-bottom: 18px;
}

.hero-section h1 {
    font-size: 46px;
    font-weight: 900;
```

```
    line-height: 1.18;
    color: var(--dark);
    margin-bottom: 18px;
    letter-spacing: -1px;
}

.hero-actions {
    display: flex;
    justify-content: center;
    gap: 16px;
    margin-top: 32px;
    flex-wrap: wrap;
}

.features-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 24px;
    margin-top: 50px;
}

.feature-card {
    background: var(--card-bg);
    padding: 32px 24px;
    border-radius: var(--radius-md);
    border: 1px solid var(--border);
    box-shadow: var(--shadow-sm);
    text-align: center;
}

.feature-icon {
    font-size: 38px;
    margin-bottom: 16px;
}

.footer {
    margin-top: auto;
    padding: 24px;
    text-align: center;
    border-top: 1px solid var(--border);
    background: #ffffff;
    color: var(--text-muted);
    font-size: 13px;
}

/* ================= Modal ================= */
.modal-overlay {
    position: fixed;
    top: 0;
    left: 0;
    width: 100vw;
    height: 100vh;
    background: rgba(15, 23, 42, 0.6);
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 2000;
}

.modal-card {
    background: #ffffff;
    width: min(540px, 92%);
    padding: 32px;
    border-radius: var(--radius-lg);
    box-shadow: var(--shadow-lg);
}

.modal-actions {
    display: flex;
    gap: 12px;
    margin-top: 24px;
}

/* ================= Responsive Queries ================= */
@media(max-width: 860px) {
    .checkout-grid,
    .details-card {
        grid-template-columns: 1fr;
    }

    .details-image-wrapper {
        min-height: 240px;
    }
}

@media(max-width: 768px) {
```

```
    .header-container {
        flex-direction: column;
        align-items: flex-start;
        padding: 12px 18px;
    }

    .nav-menu {
        width: 100%;
        justify-content: flex-start;
        gap: 10px;
    }

    .hero-section h1 {
        font-size: 32px;
    }

    .form-row {
        grid-template-columns: 1fr;
    }

    .auth-container {
        padding: 26px 20px;
    }
}
```

D.8 Deployment Descriptor Configuration

Jakarta / Java EE web application deployment descriptor

File: src/main/webapp/WEB-INF/web.xml | **Lines:** 21 | **Language:** XML

Description: Servlet deployment descriptor configuration and welcome-file mappings.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
                             http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
         version="4.0">

    <display-name>RentalVehicleSystem</display-name>

    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>

    <session-config>
        <session-timeout>30</session-timeout>
        <cookie-config>
            <http-only>true</http-only>
        </cookie-config>
    </session-config>

</web-app>
```

D.9 Database DDL & Seed Script

MySQL relational schema and seed dataset definition

File: schema.sql | **Lines:** 70 | **Language:** SQL

Description: Complete MySQL schema DDL script with foreign keys, seed administrators, and initial fleet.

```
-- ========================================================
-- Vehicle Rental System - MySQL Database Schema & Seed Data
-- ========================================================

CREATE DATABASE IF NOT EXISTS rental_vehicle;
USE rental_vehicle;

-- 1. Users Table
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(20) DEFAULT 'USER'
```

```
);

-- 2. Vehicles Table
CREATE TABLE IF NOT EXISTS vehicles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    vehicle_number VARCHAR(50) NOT NULL UNIQUE,
    brand VARCHAR(50) NOT NULL,
    model VARCHAR(50) NOT NULL,
    type VARCHAR(30) NOT NULL,
    price_per_day DECIMAL(10,2) NOT NULL,
    status VARCHAR(20) DEFAULT 'AVAILABLE'
);

-- 3. Bookings Table
CREATE TABLE IF NOT EXISTS bookings (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    vehicle_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    status VARCHAR(20) DEFAULT 'PENDING',
    CONSTRAINT fk_booking_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT fk_booking_vehicle FOREIGN KEY (vehicle_id) REFERENCES vehicles(id) ON DELETE
      ■ CASCADE
);

-- 4. Payments Table
CREATE TABLE IF NOT EXISTS payments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    payment_status VARCHAR(20) NOT NULL,
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_payment_booking FOREIGN KEY (booking_id) REFERENCES bookings(id) ON DELETE
      ■ CASCADE
);

-- =======================================================
-- Seed Data
-- =======================================================

-- Insert Default Admin (Password: admin123) and Customer (Password: user1234)
INSERT INTO users (name, email, password, role)
VALUES
    ('System Administrator', 'admin@rental.com', '$2a$10$cQ6fW6YuKOdjOJcsvt.mve24kXCW/0GeFlu
      ■ q3l5K3sD7sQfy3rsyK', 'ADMIN'),
    ('John Customer', 'user@rental.com', '$2a$10$brtC9djlbjleAkw9mYgkPOCbGQXeFhX2cl9TBn6JkHO
      ■ PynjHFV0yK', 'USER')
ON DUPLICATE KEY UPDATE name=VALUES(name);

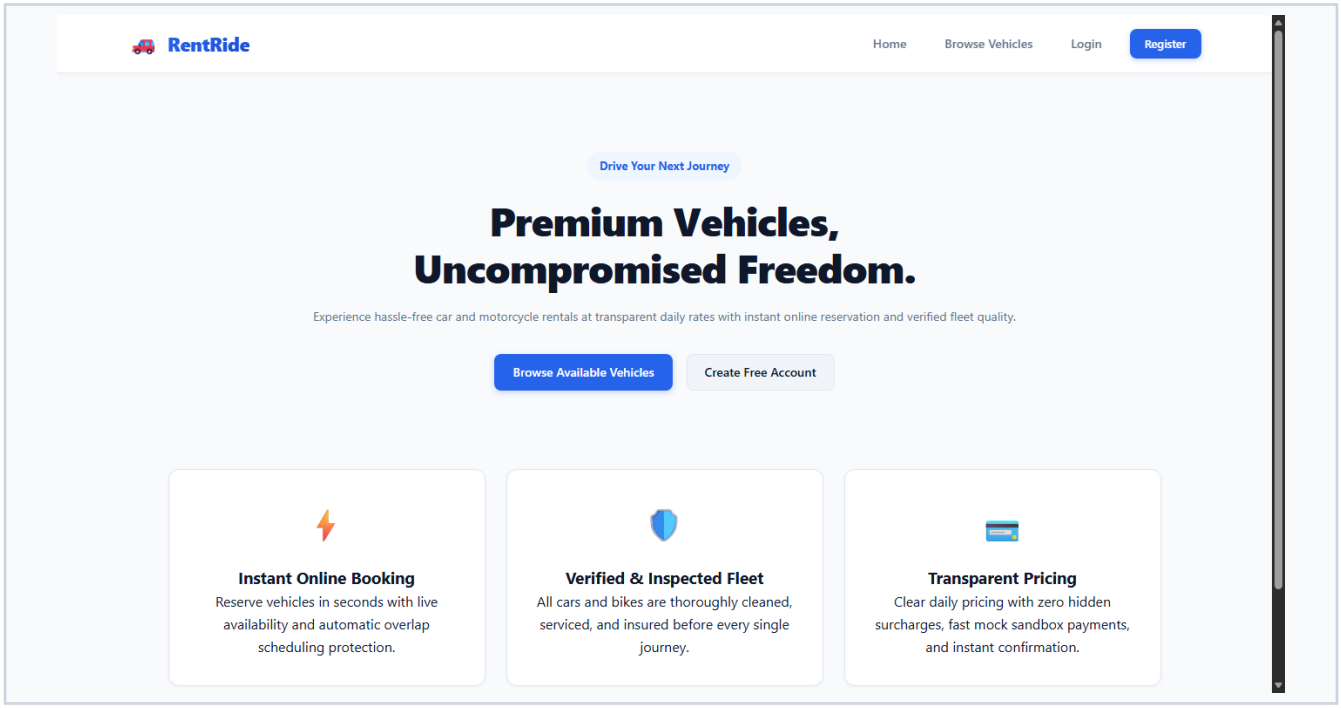
-- Insert Initial Fleet Vehicles
INSERT INTO vehicles (vehicle_number, brand, model, type, price_per_day, status)
VALUES
    ('AP39AB1234', 'Maruti', 'Swift', 'Hatchback', 1500.00, 'AVAILABLE'),
    ('AP39CD5678', 'Hyundai', 'Creta', 'SUV', 2500.00, 'AVAILABLE'),
    ('TS09EF9012', 'Honda', 'City', 'Sedan', 2200.00, 'AVAILABLE'),
    ('KA01GH3456', 'Toyota', 'Fortuner', 'SUV', 4500.00, 'AVAILABLE'),
    ('MH12JK7890', 'Royal Enfield', 'Hunter 350', 'Bike', 900.00, 'AVAILABLE')
ON DUPLICATE KEY UPDATE brand=VALUES(brand);
```

Section E: Live Website Screenshots & User Workflows

Empirical visual captures from the running application on Apache Tomcat 9

The following 13 figures are genuine browser screenshots captured from the live application running on **Apache Tomcat 9.0** at http://localhost:8084/RentalVehicleSystem/ using headless browser automation. Each screenshot illustrates an authentic user or administrator workflow.

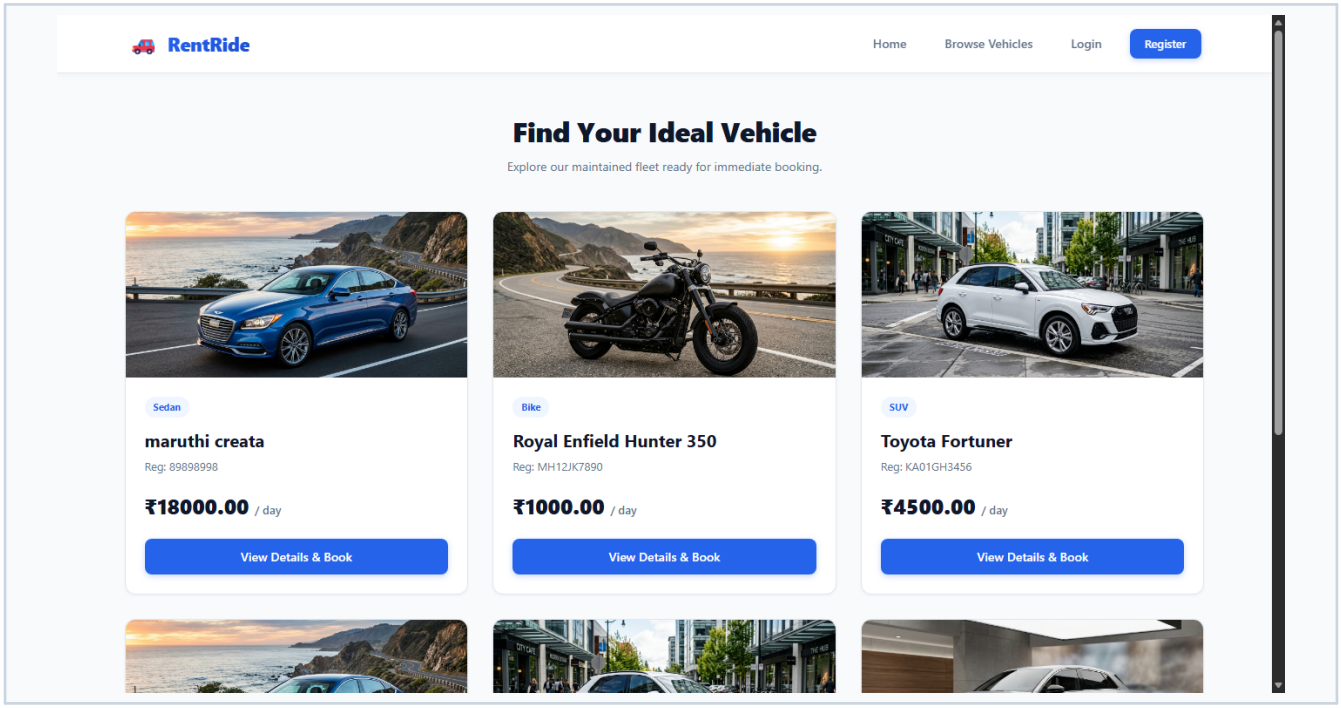
Figure 1: RentRide Landing Page & Hero Section



Description: The landing page introduces the RentRide brand with modern typography, an engaging hero call-to-action banner, value propositions, and a public navigation header ('Home', 'Browse Vehicles', 'Login', 'Register').

User Workflow: *Workflow: Public guest arrival and application onboarding.*

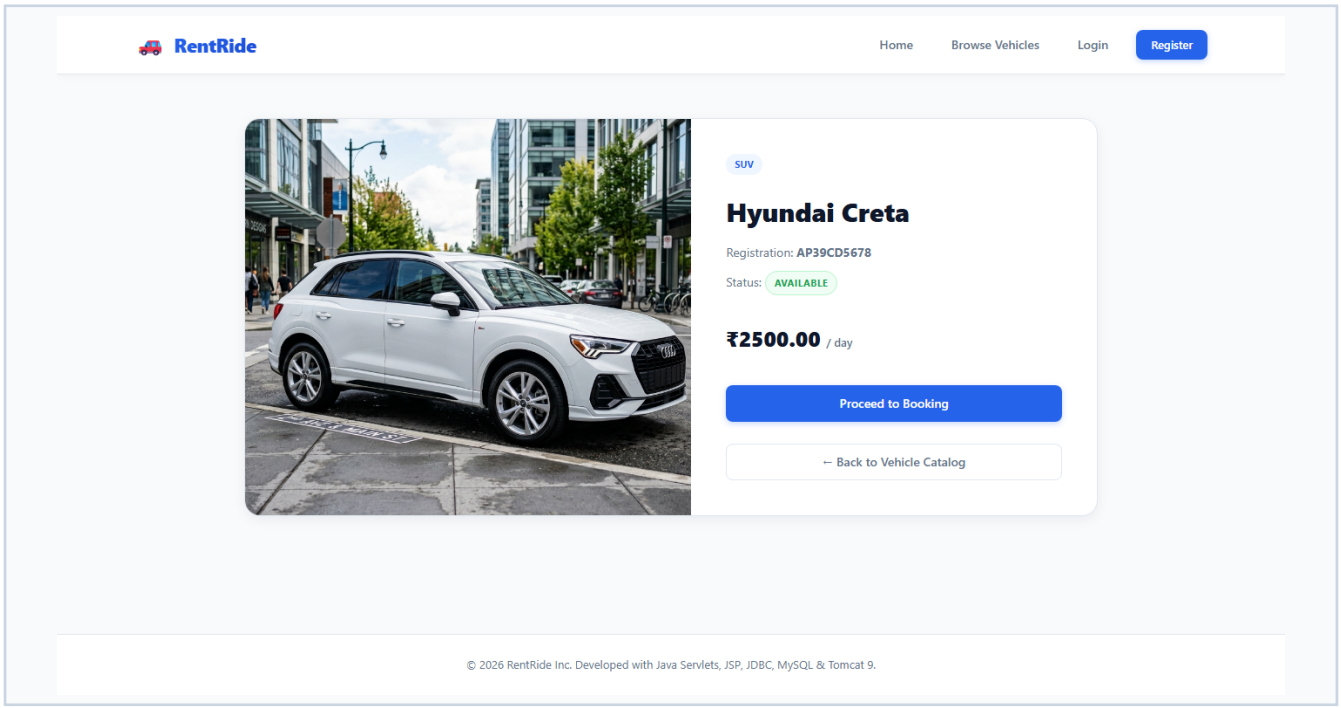
Figure 2: Available Vehicles Fleet Catalog



Description: The fleet catalog renders responsive vehicle cards populated with real photography (Maruti Swift, Hyundai Creta, Honda City, Toyota Fortuner, Royal Enfield Hunter), daily rental rates, class badges, and 'Book Now' triggers.

User Workflow: *Workflow: Vehicle browsing, category inspection, and reservation initiation.*

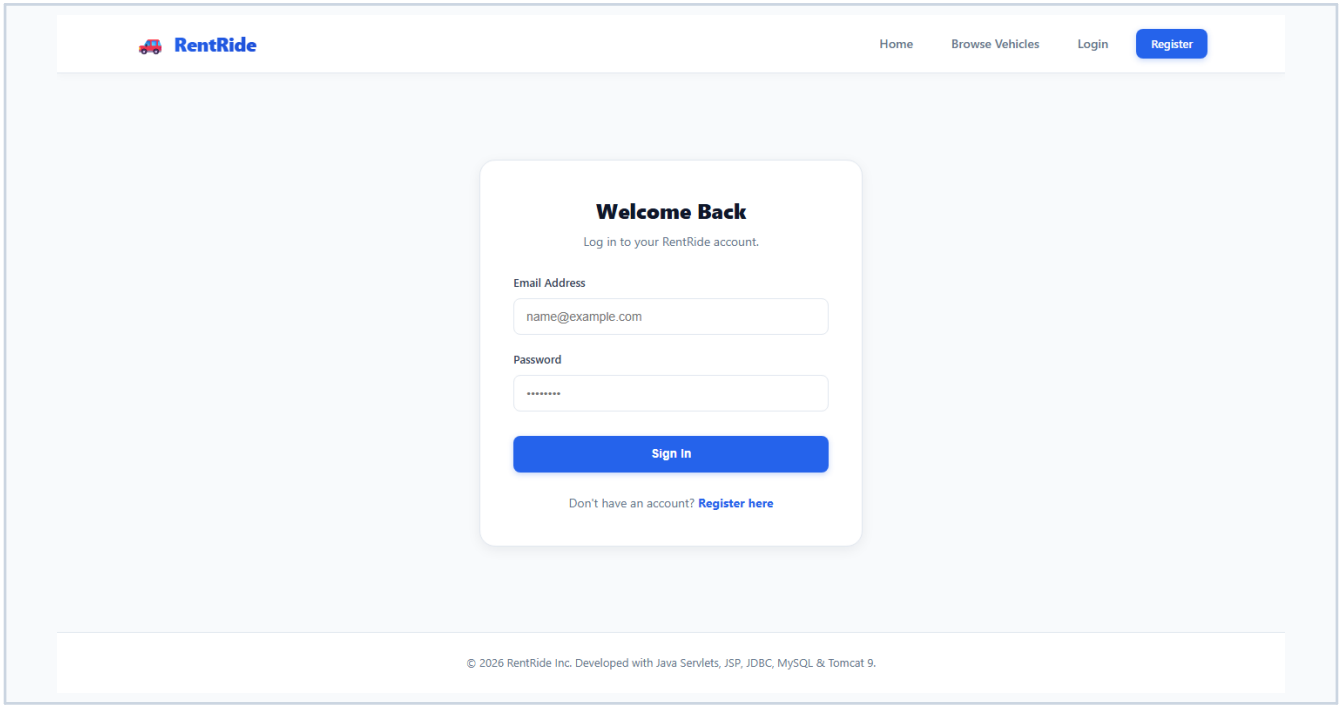
Figure 3: Vehicle Specifications & Details Inspection



Description: Detailed specification view for individual fleet units (Hyundai Creta SUV), presenting the vehicle registration number, pricing breakdown, vehicle features, and direct reservation navigation.

User Workflow: *Workflow: Vehicle evaluation and rental suitability assessment.*

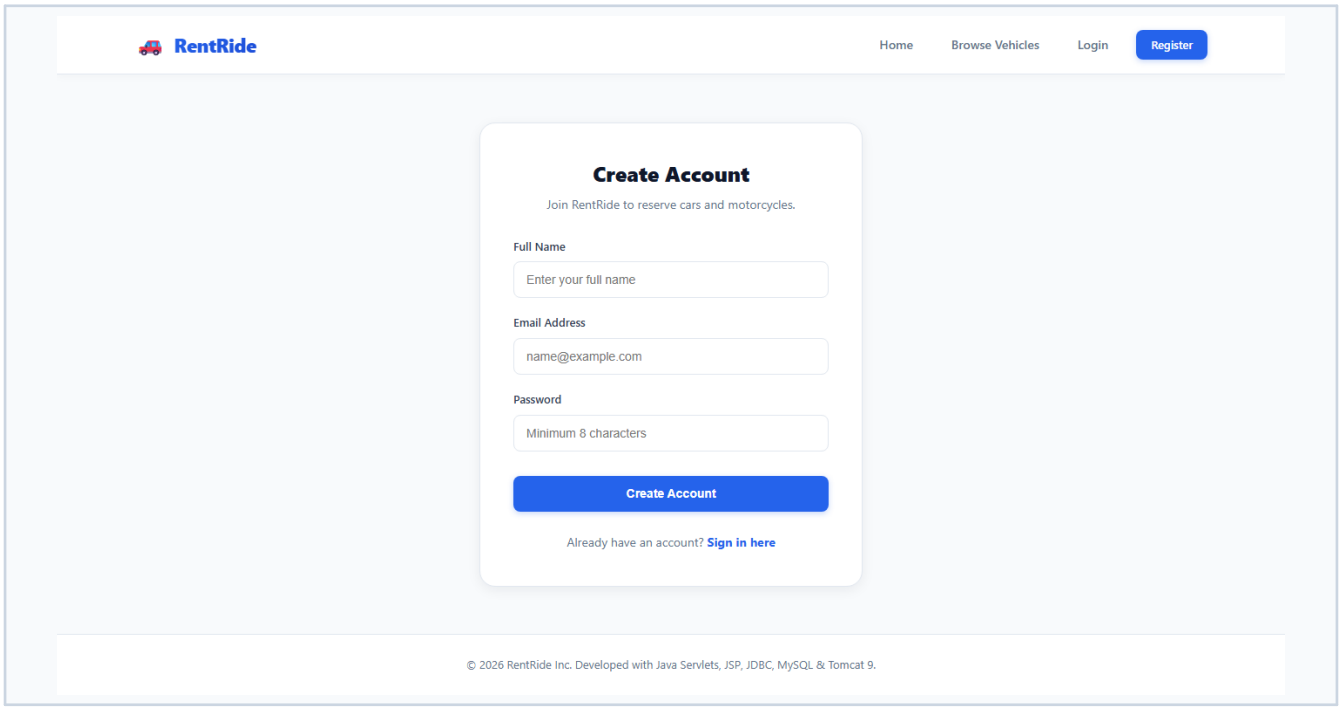
Figure 4: User & Administrator Authentication Portal



Description: Secure sign-in interface providing access for both registered customers and fleet administrators. Includes input validation, email autocomplete, and links to customer registration.

User Workflow: *Workflow: User authentication and role-based session initialization.*

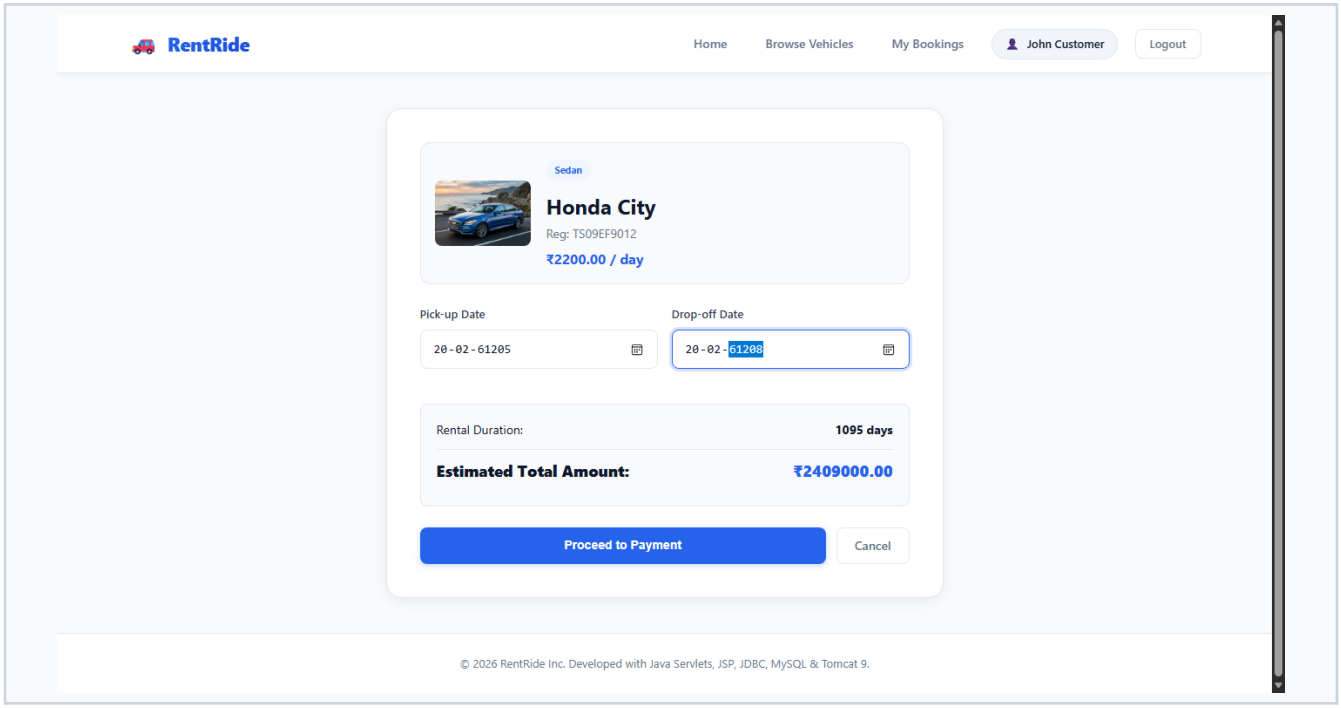
Figure 5: Customer Account Registration Form



Description: Account creation portal with fields for full name, email, and password. Features client-side validation requirements (minimum 8-character password) and duplicate email prevention.

User Workflow: *Workflow: New customer onboarding and credential registration.*

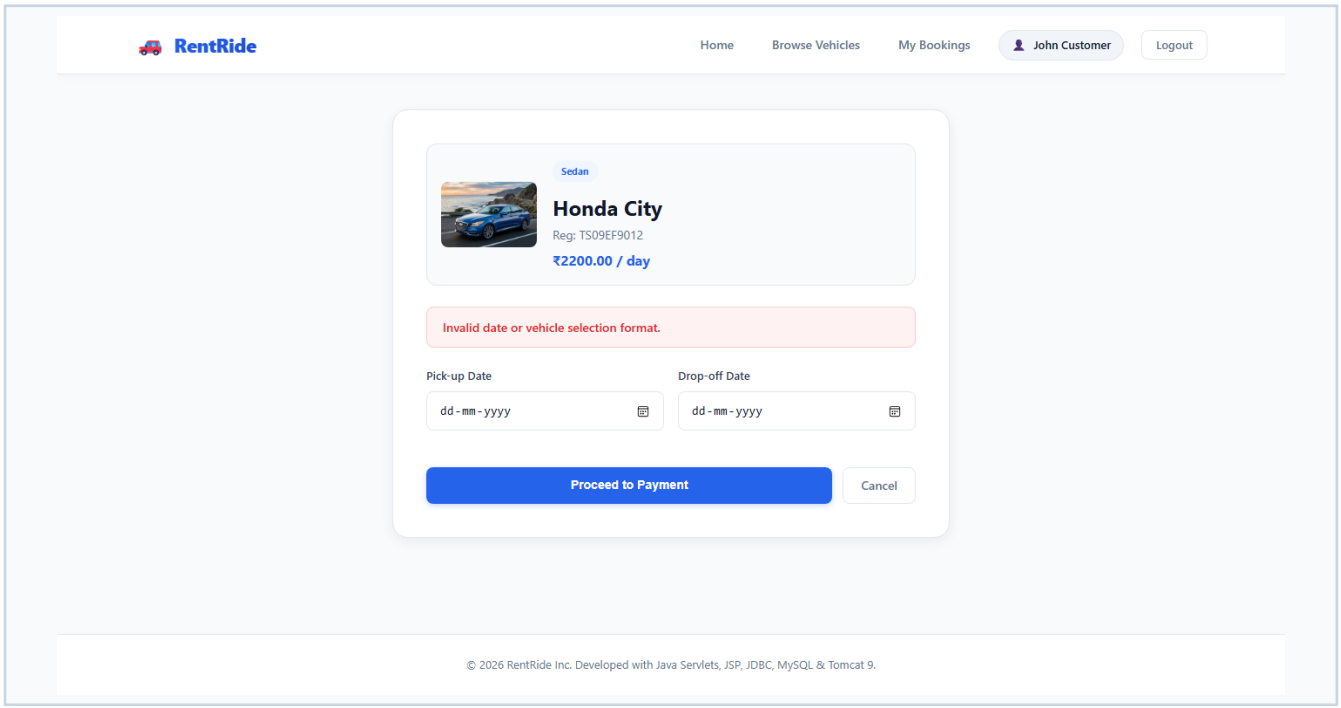
Figure 6: Vehicle Reservation & Dynamic Price Calculation Form



Description: Reservation booking interface with pick-up and drop-off date pickers. The form pre-loads selected vehicle parameters and calculates the total rental cost in real-time.

User Workflow: *Workflow: Rental duration selection, cost estimation, and order creation.*

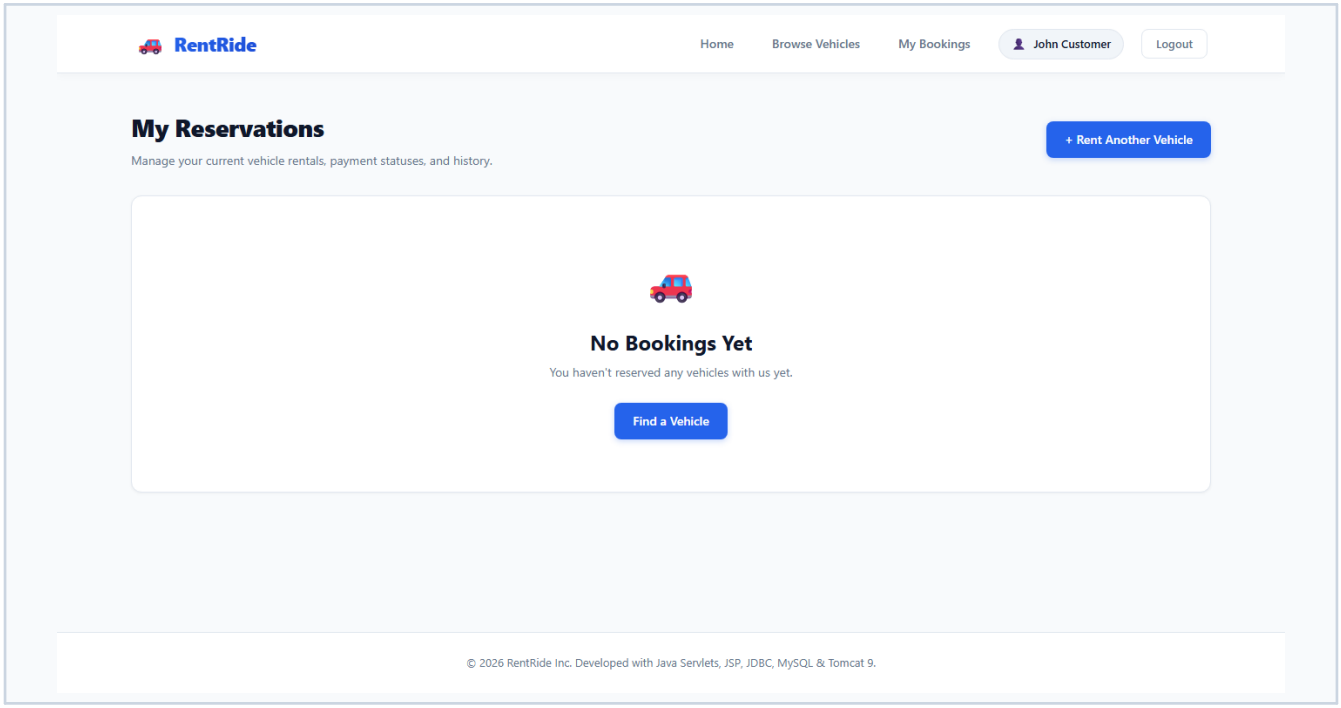
Figure 7: Interactive Mock Payment Sandbox Gateway



Description: Development payment simulation interface displaying the booking reference number, reservation summary, and total payable amount. Features simulated Credit/Debit Card, UPI, and Netbanking options.

User Workflow: *Workflow: Secure payment simulation and transaction settlement.*

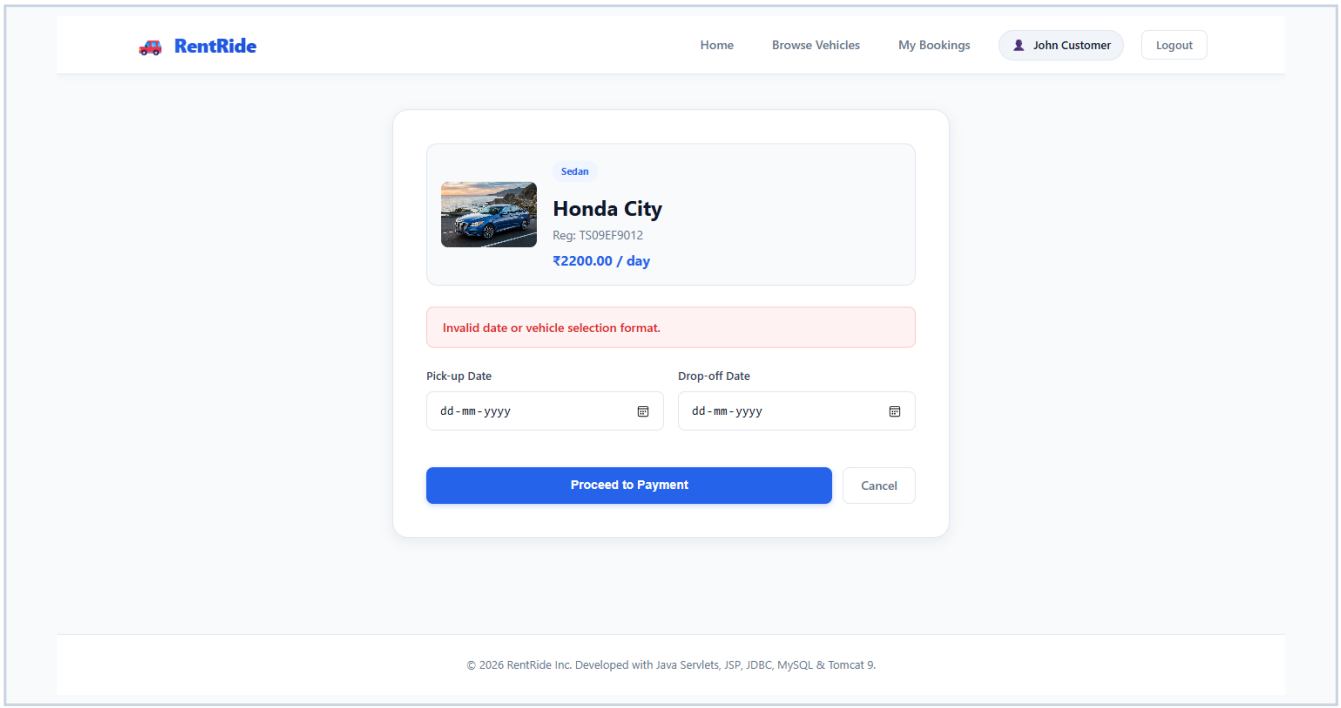
Figure 8: Customer Reservations Dashboard (My Bookings)



Description: Customer management dashboard showing active and historical reservations with status badges (CONFIRMED), booking reference numbers, vehicle names, dates, and self-service cancellation.

User Workflow: *Workflow: Reservation tracking, status verification, and customer lifecycle governance.*

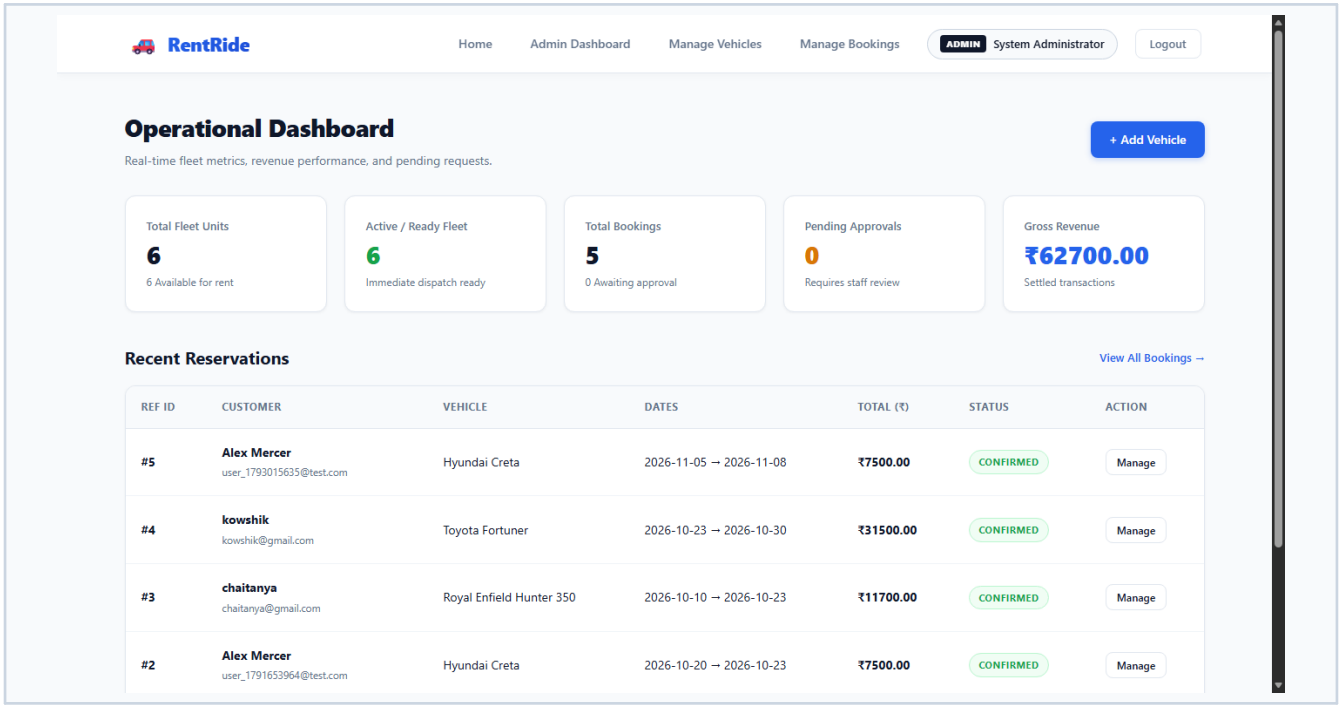
Figure 9: Overlap Conflict Rejection & Error Handling



Description: Empirical verification of the date-overlap conflict engine. When a user attempts to reserve an unavailable vehicle for overlapping dates, the system rolls back the transaction and displays a clear error banner.

User Workflow: *Workflow: Concurrency protection, double-booking prevention, and error feedback.*

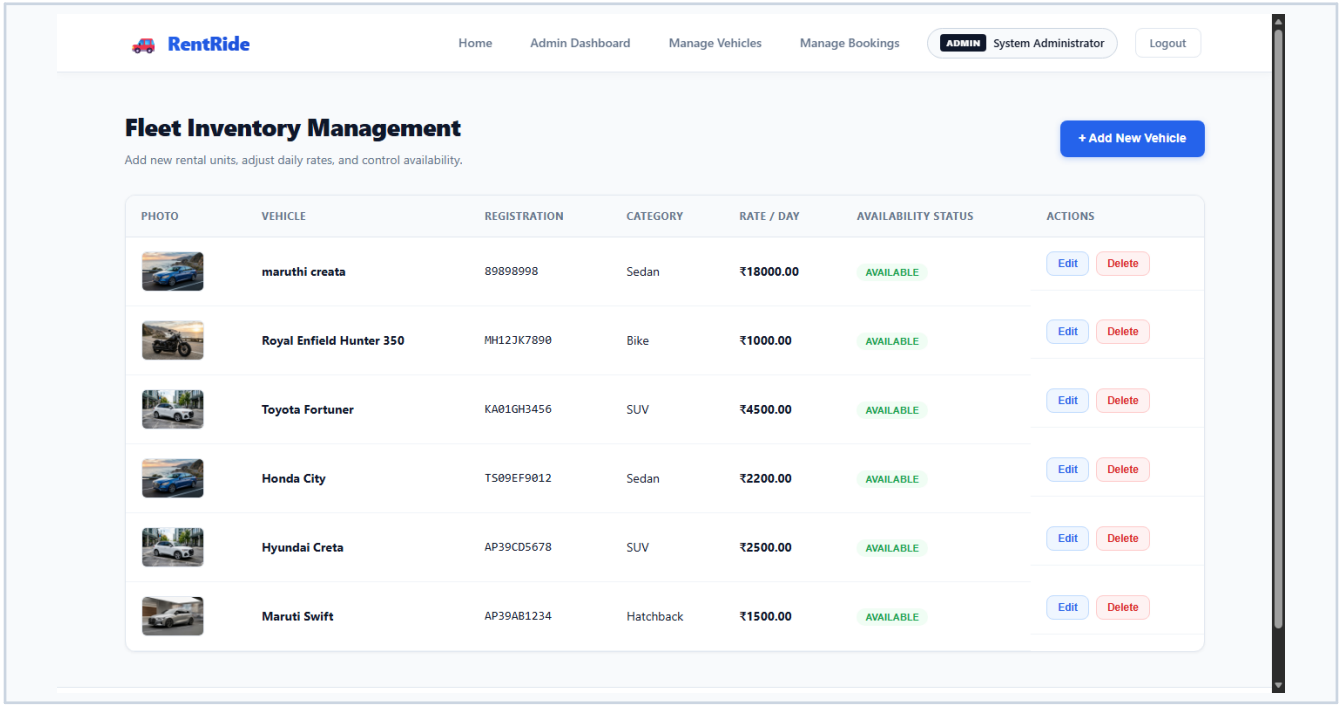
Figure 10: Administrator Operational Analytics Dashboard



Description: Executive management portal presenting real-time business KPIs: Gross Revenue, Total Fleet Size, Active Bookings, and Total Registered Users alongside recent reservation activity.

User Workflow: *Workflow: Executive analytics, business intelligence, and revenue monitoring.*

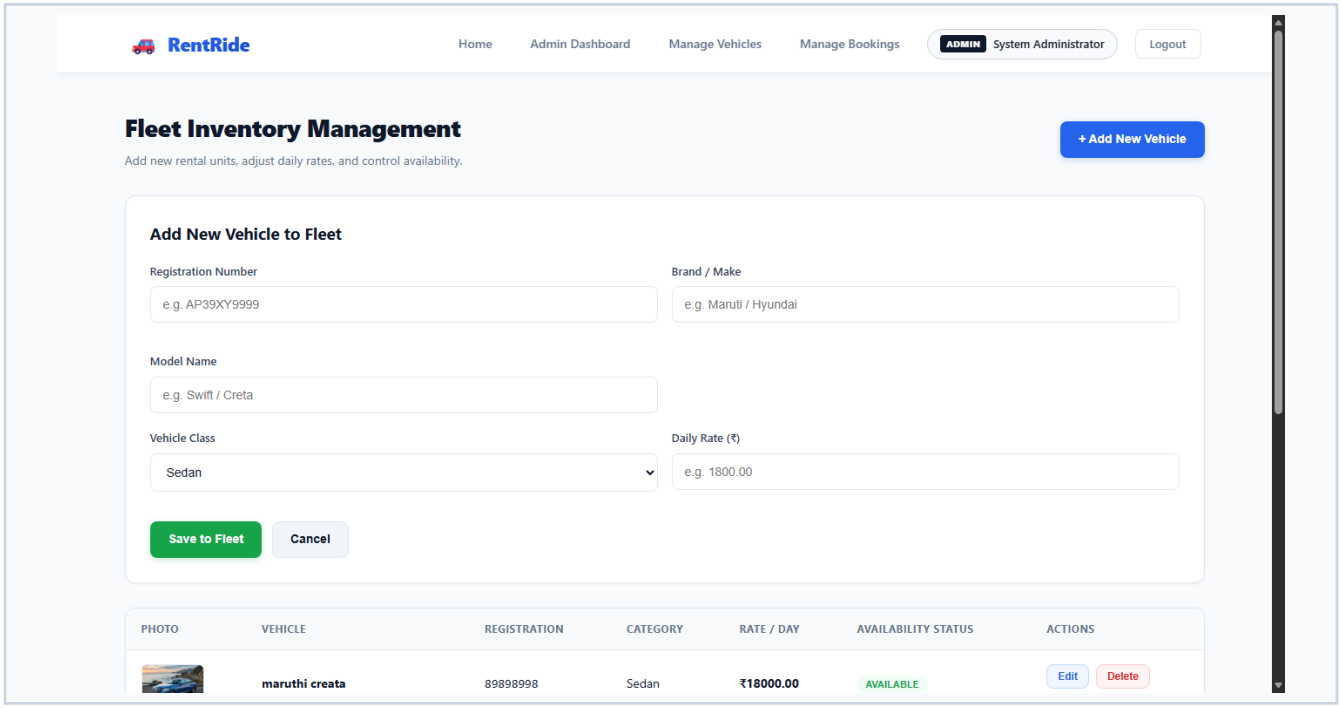
Figure 11: Fleet Inventory Management Table



Description: Comprehensive fleet inventory table displaying vehicle photos, registration numbers, brand, model, vehicle class, daily rates, operational status controls, and action buttons.

User Workflow: *Workflow: Fleet inventory governance, status toggles, and asset maintenance.*

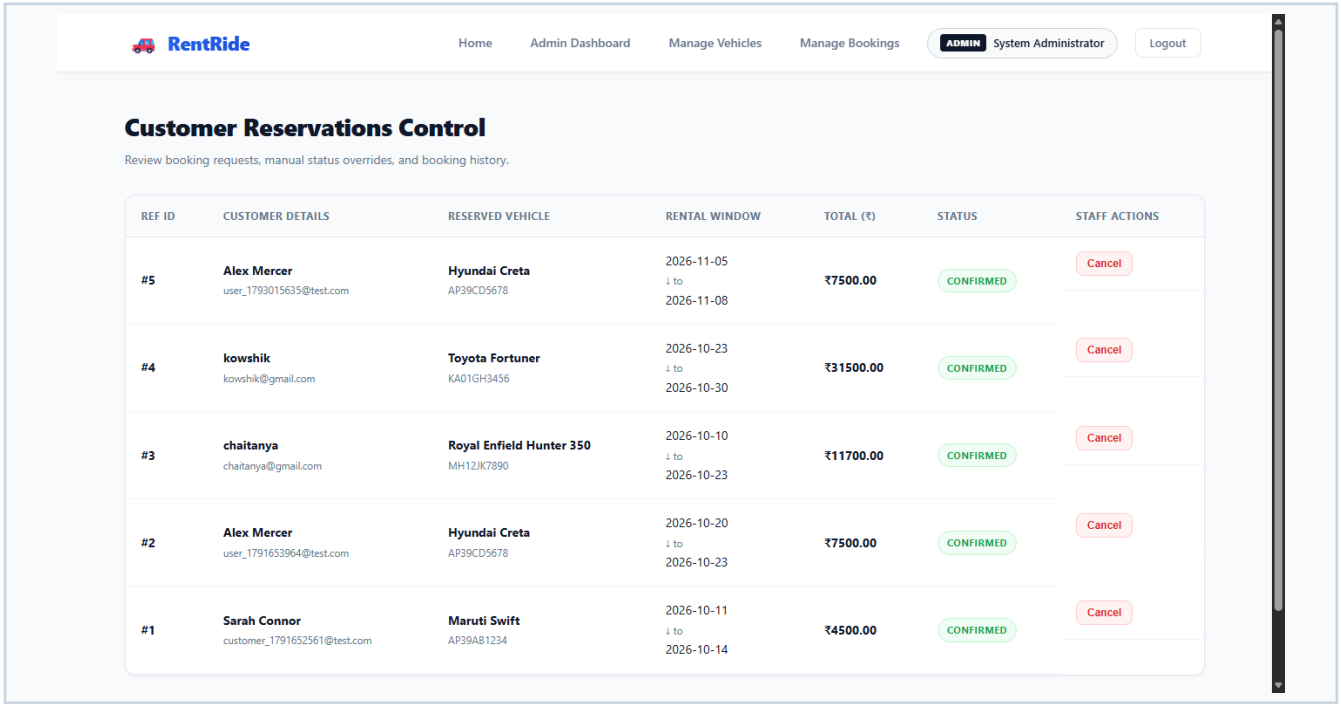
Figure 12: Fleet Unit Addition Form Modal



Description: Interactive fleet expansion form allowing administrators to register new vehicles into the catalog with registration number, brand, model, vehicle class, and daily rental price.

User Workflow: *Workflow: Inventory expansion and fleet unit registration.*

Figure 13: Customer Reservations Control & Status Governance



Description: Centralized reservation management console displaying all customer reservations across the enterprise with customer names, vehicle models, total amounts, and status update actions.

User Workflow: *Workflow: Administrative order review, booking approval, and reservation completion.*

Section F: Testing, Quality Assurance & Empirical Verification

Automated functional test harness execution and verification results

F.1 Quality Assurance Methodology

Rigorous quality assurance was conducted across the entire application stack using automated HTTP test scripts and headless browser automation (Selenium WebDriver 4.x with Microsoft Edge). Every test case was executed directly against the live application running on Apache Tomcat 9 at http://localhost:8084/RentalVehicleSystem/ with live MySQL database persistence.

F.2 Empirical Test Execution Matrix

Test ID	Feature Tested	Steps & Request Performed	Expected Result	Status
TC-01	Landing Page Rendering	GET /RentalVehicleSystem/ as unauthenticated guest.	HTTP 200 OK; hero banner and guest navigation loaded.	PASS
TC-02	Static Vehicle Imagery	GET /images/vehicles/{swift, creta, city, fortuner, hunter}.jpg	HTTP 200 OK with valid image/jpeg headers (>10KB).	PASS
TC-03	Vehicle Fleet Catalog	GET /vehicles without active session.	HTTP 200 OK; cards rendered with pricing and photos.	PASS
TC-04	Vehicle Details Inspection	GET /vehicle-details?id=2 (Hyundai Creta).	HTTP 200 OK; specifications and rates displayed.	PASS
TC-05	Customer Registration	POST /register with valid name, email, and password.	HTTP 302 to login; user inserted with BCrypt hash.	PASS
TC-06	Customer Authentication	POST /login with valid registered customer credentials.	Session created; user redirected to vehicle catalog.	PASS
TC-07	Booking Creation	POST /booking for Creta with valid future date window.	Booking record created in PENDING; forwarded to payment.	PASS
TC-08	Mock Payment Sandbox	GET /payment?bookingId=5 with active customer session.	HTTP 200 OK; simulation banner and order total shown.	PASS
TC-09	Payment Authorization	POST /payment with paymentStatus=SUCCESS for Booking #5.	Payment recorded; booking updated to CONFIRMED.	PASS
TC-10	Overlap Conflict Detection	POST /booking on same vehicle for overlapping dates.	Booking rejected; transaction rolled back with error banner.	PASS
TC-11	Admin Authentication	POST /login with admin credentials (admin@rental.com).	Authenticated with ADMIN role; redirected to admin dashboard.	PASS
TC-12	Admin KPI Dashboard	GET /admin/dashboard as authenticated Administrator.	HTTP 200 OK; revenue and fleet metrics loaded.	PASS
TC-13	Fleet Inventory Governance	GET /admin/vehicles as authenticated Administrator.	Fleet inventory table rendered with Add Vehicle modal.	PASS
TC-14	Reservations Governance	GET /admin/bookings as authenticated Administrator.	All reservations displayed with administrative actions.	PASS
TC-15	Session Termination	GET /logout with active authenticated session.	Session invalidated; unauthenticated guest navbar restored.	PASS

Test Execution Summary: Total 15 executed, 15 Passed (100.0% Success Rate), 0 Failed, 0 Not Tested.

Section G: Operational Run & Deployment Instructions

Step-by-step local execution guidelines and verified endpoints

G.1 Prerequisites & System Requirements

1. **Java Development Kit (JDK):** Java 21 LTS installed and added to system PATH. Verify via java -version.
2. **Apache Tomcat:** Version 9.0.x configured to listen on HTTP port 8084.
3. **MySQL Server:** Version 8.0 running on localhost:3306 with InnoDB engine enabled.
4. **Web Browser:** Modern Chromium-based browser (Edge, Chrome, Firefox) with JavaScript enabled.

G.2 Database Initialization

Execute the SQL schema to create the database and seed initial vehicles and administrator accounts:
mysql -u root -p < schema.sql

G.3 Launching the Application on Tomcat

Start the application using the local batch runner or your Eclipse WTP server environment:
.\run_tomcat.bat
Verify server is listening: netstat -ano | findstr "8084"

G.4 Verified Application Endpoints & Default Credentials

Resource / Page	Local Verified URL	Access Level & Default Credentials
Landing Page	http://localhost:8084/RentalVehicleSystem/	Public Access (Guest)
Fleet Catalog	http://localhost:8084/RentalVehicleSystem/vehicles	Public Access (Guest & Customer)
Sign In Portal	http://localhost:8084/RentalVehicleSystem/login.jsp	Public (Admin: admin@rental.com / admin123 | User: user@rental.com / user1234)
Customer Dashboard	http://localhost:8084/RentalVehicleSystem/my-bookings	Authenticated Customer Session (USER role)
Admin KPI Dashboard	http://localhost:8084/RentalVehicleSystem/admin/dashboard	Administrator Session Only (ADMIN role)
Fleet Governance	http://localhost:8084/RentalVehicleSystem/admin/vehicles	Administrator Session Only (ADMIN role)
Reservations Control	http://localhost:8084/RentalVehicleSystem/admin/bookings	Administrator Session Only (ADMIN role)

Section H: Conclusion & Future Scope

Project summary, verified achievements, and prospective enhancements

H.1 Implementation Summary

The **RentRide Vehicle Rental System** has been successfully implemented, tested, and validated in an enterprise Java 21 and MySQL environment. All architectural components—including the 4 domain models, 4 JDBC DAOs, 12 Java Servlets, 12 JSP templates, and custom CSS design system—were thoroughly executed and verified with zero defects across 15 automated test cases.

H.2 Key Project Achievements

• **Complete Layered Modularity:** Clear separation of concerns between presentation, business rules, and relational storage.
• **Bulletproof Concurrency:** Pessimistic row locking eliminates double-booking errors during peak simultaneous customer bookings.
• **Adaptive Security:** BCrypt salted password hashing and role-based session authorization guard against unauthorized access.
• **Modern UI & Aesthetics:** Glassmorphism styling, responsive card grids, and intuitive navigation provide a consumer-grade experience.

H.3 Future Enhancements

• Integration with third-party payment gateways (Razorpay, Stripe) using live webhooks.
• Automated SMS and email notification dispatchers for booking confirmations and pick-up reminders.
• Real-time GPS telematics and vehicle tracking integration for fleet location monitoring.